

Estatística Computacional

Departamento de Estatística e Matemática Aplicada/CC/UFC

Laboratório de Inovação em Estatística - StatLab

Grupo de pesquisa Modelos de Sobrevida e Aplicações.

Versão Preliminar
- Em Elaboração -



Índice

Informações Legais e Declaração de Uso de Inteligência Artificial	4
Direitos Autorais e Uso da Obra	4
Declaração sobre o Uso de Ferramentas de Inteligência Artificial	4
Prefácio	5
Identificação da Disciplina	6
EMENTA	6
OBJETIVO GERAL	6
OBJETIVOS ESPECÍFICOS	7
CONTEÚDO PROGRAMÁTICO	7
Unidade I – Introdução ao R e programação estatística	7
Unidade II – Geração de números pseudoaleatórios	8
Unidade III – Simulação de variáveis aleatórias	8
Unidade IV – Métodos de Monte Carlo	8
Unidade V – Bootstrap e Jackknife	9
Unidade VI – Otimização numérica	9
Unidade VII – Algoritmo EM	10
Unidade VIII – Aplicações modernas em Estatística Computacional	10
METODOLOGIA	10
AVALIAÇÃO	11
CRITÉRIOS DE AVALIAÇÃO	11
BIBLIOGRAFIA BÁSICA	11
BIBLIOGRAFIA COMPLEMENTAR	12
Observações Finais	12
1 Introdução ao R e Programação Estatística	13
1.1 Motivação	14
Da teoria à simulação	14
Um atalho analítico útil	14
O papel da simulação	15
Atividade: Problema do Aniversário (22 jogadores)	16
Exercícios	17
1.2 Uma Pequena Introdução ao R	17

Instalação	18
Windows	18
macOS	19
Linux	19
1.3 O que é o R	23
1.4 Números, Aritmética, Atribuição e Vetores	43
Exercícios	49
1.5 Matrizes	50
Exercícios	52
1.6 Operações com matrizes	54
Exercícios	58
1.7 Laços e repetições	59
1.8 Escrevendo funções no R	62
1.8.1 Média Aritmética	63
1.8.2 Mediana	63
Exercícios	64
2 Geração de Números Pseudoaleatórios	65
2.1 Geração de sequências $U(0, 1)$	65
2.2 Por que “Pseudoaleatórios”?	66
2.3 O Gerador Congruencial Linear (GCL)	66
2.3.1 Escolha de Parâmetros e Período	66
2.4 Implementação no R	67
Por que o GCL funciona?	67
2.5 Como validar um Gerador?	69
Limitações	69
3 Simulação de Variáveis Aleatórias	72
3.1 Caso Discreto	72
3.1.1 Método da Transformação Inversa	72
Exemplo 1	73
Exemplo 2	76
Exemplo 3	77
Exemplo 4	78
Exercício 5	79
3.1.2 Método da Aceitação-Rejeição	79
3.1.3 Método da Composição	83
3.2 Caso Contínuo	84
3.2.1 Método da Transformação Inversa	84
3.2.2 Método da Aceitação e Rejeição	84
3.2.3 Método da Composição (Misturas)	85

4	Métodos de Monte Carlo	86
4.1	Uso de Números Aleatórios na Avaliação de Integrais	87
	Estimativa de Probabilidades via Monte Carlo	91
4.1.1	Erro e Convergência de Monte Carlo	92
4.2	Simulações de Monte Carlo em Inferência	94
4.2.1	Instalação	95
4.3	Execute sua primeira Simulação de Monte Carlo com <code>future_mc()</code>	95
5	Bootstrap e Jackknife	99
5.1	Bootstrap	99
5.1.1	Acurácia da média amostral	100
5.1.2	Estimativa bootstrap do erro padrão	101
5.1.3	Bootstrap Paramétrico	105
5.1.4	Intervalos de Confiança	106
5.2	O Método Jackknife	112
5.2.1	O Princípio “Leave-One-Out”	112
5.2.2	Estimativa de Viés e Variância	112
5.2.3	Exemplos Computacionais em R	113
5.2.4	Jackknife vs. Bootstrap	115
6	Otimização Numérica	116
6.1	Método de Newton	116
7	Algoritmo EM	123
8	Aplicações Modernas em Estatística Computacional	124
	Referências	125

Informações Legais e Declaração de Uso de Inteligência Artificial

Direitos Autorais e Uso da Obra

© 2026 — Os autores.

Todos os direitos reservados. Esta obra é protegida pela legislação brasileira de direitos autorais (Lei nº 9.610/1998). É vedada a reprodução, distribuição, armazenamento ou transmissão total ou parcial deste livro, por qualquer meio ou processo, eletrônico ou mecânico, sem autorização expressa e por escrito dos autores, salvo nos casos permitidos pela legislação vigente para fins exclusivamente acadêmicos e não comerciais, com a devida citação da fonte.

A utilização do material em ambientes de ensino é permitida desde que preservada a integridade do conteúdo, mencionada a autoria e respeitados os princípios de uso responsável da informação científica. A reprodução para fins comerciais, bem como a modificação substancial do conteúdo sem autorização, constitui violação de direitos autorais.

Declaração sobre o Uso de Ferramentas de Inteligência Artificial

Durante a elaboração deste livro foram utilizadas ferramentas de Inteligência Artificial, em especial sistemas de apoio à escrita e organização textual, como recurso complementar no processo de produção acadêmica.

O uso de IA teve caráter **estritamente assistivo**, não substituindo o desenvolvimento conceitual, a formulação matemática, a interpretação estatística nem as decisões pedagógicas da obra. Todo conteúdo foi cuidadosamente revisado, validado e adaptado pelos autores, que assumem integral responsabilidade científica e intelectual pelo texto final.

Não foram delegadas à IA decisões metodológicas, inferenciais ou conclusões analíticas. A elaboração teórica, a seleção de exemplos, a validação matemática e a coerência didática resultam da experiência acadêmica e da atuação dos autores na área de Modelos de Regressão.

Reafirmamos que o uso responsável de tecnologias emergentes deve sempre preservar o rigor científico, a integridade acadêmica e o protagonismo intelectual humano.

Prefácio

Este livro resulta da experiência acumulada no ensino, na orientação de estudantes e no desenvolvimento de pesquisas em modelos de regressão. Somos docentes pesquisadores do Departamento de Estatística e Matemática Aplicada da Universidade Federal do Ceará (DEMA/UFC), com atuação contínua em regressão linear, modelagem estatística e inferência. Integramos o Laboratório de Inovação em Estatística — StatLab/UFC (<https://statlab.quarto.pub/>) e o Grupo de Pesquisa *Modelos de Sobrevivência e Aplicações* do CNPq (<http://dgp.cnpq.br/dgp/espelhogrupo/9639095385610769>), espaços nos quais articulamos ensino, pesquisa e aplicação a problemas reais.

Este livro resulta de anos de experiência em sala de aula dos professores Ronald Targino, Rafael Braz, Juvêncio Nobre e Manoel Santos-Neto. Destina-se a apoiar os alunos da graduação em Estatística e do Programa de Pós-Graduação em Modelagem e Métodos Quantitativos (PPGMMQ) do Departamento de Estatística e Matemática Aplicada (DEMA) da Universidade Federal do Ceará (UFC).

Ao longo dos capítulos, abordamos a geração de números aleatórios (discretos e contínuos); métodos de suavização; simulação estocástica por inversão, rejeição e composição, bem como métodos de reamostragem; métodos de aproximação e integração; quadratura Gaussiana, integração de Monte Carlo e quadratura adaptativa; métodos de Monte Carlo em sentido amplo; amostradores MCMC, com ênfase em Gibbs e Metropolis–Hastings; otimização numérica via Newton–Raphson, Fisher scoring e quase-Newton, além do algoritmo EM; Bootstrap e Jackknife; diagnóstico de convergência; e aspectos computacionais em problemas práticos, com foco em implementação eficiente, estabilidade numérica e reprodutibilidade dos resultados.

Esperamos que este material sirva não apenas como texto-base para as disciplinas Estatística Computacional (graduação em Estatística) e Métodos Computacionais em Estatística (Mestrado-PPGMMQ), mas também como suporte para aqueles que desejam programar com qualidade na área de Estatística.

Identificação da Disciplina

- Curso: Estatística
- Disciplina: Estatística Computacional
- Carga Horária: 64 horas
- Créditos: 4
- Regime: Semestral
- Pré-requisitos: Probabilidade II (CC0285) e Análise Exploratória de Dados (CC0280) e Sistemas de Informação e Banco de Dados (CK0084)
- Período Letivo: 2026.2
- Professor(a): Dr. Manoel Santos-Neto

EMENTA

Uso de pacotes estatísticos: R ou SPLUS, SPSS. Planilhas eletrônicas com apoio ferramental. Geração números pseudo-Aleatórios e variáveis aleatórias. Simulação estatística: métodos de inversão, rejeição, composição e métodos de reamostragem. Otimização numérica: Newton-Raphson, scoring, quase-Newton. Algoritmo EM. “Bootstrap” e “Jackknife”. Métodos de Monte Carlo.

OBJETIVO GERAL

Desenvolver competências teóricas e computacionais para compreensão, implementação e aplicação dos principais métodos de Estatística Computacional, utilizando a linguagem R como ferramenta principal.

OBJETIVOS ESPECÍFICOS

- Utilizar o R para programação e análise estatística;
- compreender e implementar métodos de geração de números pseudoaleatórios e variáveis aleatórias;
- aplicar técnicas de simulação e métodos de Monte Carlo;
- compreender e aplicar os métodos Bootstrap e Jackknife;
- implementar métodos de otimização numérica e o algoritmo EM;
- avaliar criticamente a precisão, convergência e eficiência dos métodos computacionais;
- desenvolver análises estatísticas computacionais organizadas e reproduzíveis.

CONTEÚDO PROGRAMÁTICO

Unidade I – Introdução ao R e programação estatística

- Estatística Computacional: conceitos, objetivos e aplicações;
- ambiente computacional e organização de projetos em R;
- objetos, vetores, matrizes, listas e data.frame;
- estruturas condicionais e estruturas de repetição;
- funções e programação modular;
- programação vetorizada e noções de programação funcional;
- manipulação e análise de dados;
- organização, documentação e reprodutibilidade do código;
- introdução ao desenvolvimento de relatórios computacionais reproduzíveis.

Unidade II – Geração de números pseudoaleatórios

- aleatoriedade e pseudoaleatoriedade;
- princípios dos geradores de números pseudoaleatórios;
- geradores congruenciais;
- período e propriedades desejáveis de um gerador;
- sementes e reprodutibilidade;
- geração de números pseudoaleatórios no R;

Unidade III – Simulação de variáveis aleatórias

- fundamentos da simulação estatística;
- geração de variáveis aleatórias discretas e contínuas;
- método da transformação inversa;
- método da rejeição;
- método da composição;
- implementação dos métodos em R;

Unidade IV – Métodos de Monte Carlo

- fundamentos dos métodos de Monte Carlo;
- Lei dos Grandes Números e fundamentação probabilística;
- estimação de probabilidades e esperanças por simulação;
- integração de Monte Carlo;
- introdução às técnicas de redução de variância;
- planejamento e implementação de estudos de simulação; aplicações estatísticas em R.

Unidade V – Bootstrap e Jackknife

- princípios dos métodos de reamostragem;
- distribuição empírica;
- Bootstrap não paramétrico;
- Bootstrap paramétrico;
- estimação Bootstrap de viés e erro padrão;
- intervalos de confiança Bootstrap;
- método Jackknife;
- estimação de viés e erro padrão por Jackknife;
- análise da influência das observações;
- comparação entre Bootstrap e Jackknife;
- limitações dos métodos de reamostragem;
- implementação e aplicações em R.

Unidade VI – Otimização numérica

- problemas de otimização em Estatística;
- funções objetivo e log-verossimilhança;
- métodos iterativos e critérios de convergência;
- método de Newton-Raphson;
- função score e matriz Hessiana;
- informação observada e informação de Fisher;
- método Score de Fisher;
- métodos quase-Newton;
- método BFGS e noções de L-BFGS;
- escolha de valores iniciais;
- estabilidade e diagnóstico de convergência;
- utilização de funções de otimização disponíveis no R;
- aplicações à estimação por máxima verossimilhança.

Unidade VII – Algoritmo EM

- problemas estatísticos envolvendo informações incompletas e variáveis latentes;
- dados observados e dados completos;
- estrutura geral do algoritmo EM;
- etapa da esperança (E);
- etapa da maximização (M);
- propriedade de monotonicidade da log-verossimilhança;
- critérios de convergência;
- sensibilidade aos valores iniciais;
- máximos locais e limitações computacionais;
- implementação do algoritmo EM em R;
- aplicação a modelos de mistura de distribuições;
- interpretação e diagnóstico dos resultados.

Unidade VIII – Aplicações modernas em Estatística Computacional

- planejamento de estudos computacionais;
- comparação empírica de estimadores e procedimentos estatísticos;
- avaliação de viés, variância, erro quadrático médio e cobertura;
- reprodutibilidade em experimentos computacionais;
- documentação e comunicação de resultados.

METODOLOGIA

A disciplina será desenvolvida por meio de aulas teóricas e práticas, integrando fundamentos estatísticos, formulação matemática e implementação computacional em R. Serão utilizados exemplos, exercícios, estudos de simulação e atividades computacionais, com organização didática que favoreça diferentes formas de aprendizagem, preservando o rigor teórico e computacional.

AVALIAÇÃO

A avaliação será contínua, por meio de avaliações individuais, atividades computacionais, exercícios e/ou projetos. Serão considerados tanto o domínio dos fundamentos teóricos quanto a capacidade de implementar, analisar e interpretar os métodos estudados.

CRITÉRIOS DE AVALIAÇÃO

Serão considerados:

- domínio dos conceitos teóricos;
- correção estatística e computacional;
- qualidade da implementação em R;
- interpretação adequada dos resultados;
- clareza e organização das análises;
- capacidade de avaliar limitações, convergência e estabilidade dos métodos.

BIBLIOGRAFIA BÁSICA

1. ROSS, SHELDON. Simulation, 4th ed., San Diego, California: Elsevier, 2006
2. EFRON, B.; TIBSSHIRANI, R. J. An Introducion to the Bootstrap. New York: Chapman e Hall, 1993.
3. LIBREOFFICE. Introdução ao CALC, Planilhas de cálculo no LibreOffice. Disponível em: <<https://documentation.libreoffice.org/assets/Uploads/Documentation/pt-br/CG70/CG70-CalcGuide-Master.pdf>> Acesso em: 15 mar. 2022.
4. ROBERT, C; CASELLA, G. Introducing Monte Carlo Methods with R. Springer eBooks XX, 284 p (Use R). ISBN 9781441915764. Disponível em 86 <https://doi.org/10.1007/978-1-4419-1576-4> Acesso em : 15 mar. 2022.
5. CRAWLEY, Michael J. The R Book. Chichester: John Wiley & Sons, 2007.

BIBLIOGRAFIA COMPLEMENTAR

1. DAVIDSON, A. C.; HINKLEY, D. V. Bootstrap Methods and their Application. New York: Cambridge University Press,
2. AQUINO, J. ALVES. R para cientistas sociais. <http://www.uesc.br/editora/livrosdigitais_20140513/r_cientistas.pdf>
3. GARRET G.; HADLEY W. R for Data Science. Disponível em: <https://r4ds.had.co.nz/> Acesso em : 15 mar. 2022.
4. DALGAARD, P. Introductory Statistics with R. Springer, 2008.
5. BRUNI, Adriano Leal. SPSS aplicado à pesquisa acadêmica. São Paulo: Atlas, 2009.
6. KELLER, A; HEINRICH, S; NIEDERREITER, H. SPRINGERLINK (ONLINE SERVICE). Monte Carlo and Quasi-Monte Carlo Methods 2006. Springer eBooksBerlin, Heidelberg: Springer-Verlag Berlin Heidelberg, 2008. ISBN Disponível em: <http://dx.doi.org/10.1007/978-3-540-74496-2> Acesso em: 15 mar. 2022

Observações Finais

A linguagem R será utilizada como principal ambiente computacional da disciplina. Os materiais e atividades buscarão integrar teoria e prática, com atenção à acessibilidade pedagógica, à organização dos conteúdos e à reprodutibilidade das análises. O cronograma poderá ser ajustado conforme o desenvolvimento da turma e o calendário acadêmico.

1 Introdução ao R e Programação Estatística

A simulação tem um papel preponderante na estatística moderna, e suas vantagens no ensino de Estatística são conhecidas há muito tempo. Em um de seus primeiros números, o periódico *Teaching Statistics* publicou artigos que aludem precisamente a isso. Thomas; Moore (1980) afirmaram que “a introdução do computador na sala de aula escolar trouxe uma nova técnica para o ensino, a técnica da simulação”. Zieffler; Garfield (2007) e Tintle et al. (2015) discutem o papel e a importância da aprendizagem baseada em simulação no currículo de graduação em Estatística. No entanto, outros autores (por exemplo, Hodgson; Burke, 2000) discutem alguns problemas que podem surgir ao ensinar uma disciplina por meio de simulação, a saber, o desenvolvimento de certos equívocos na mente dos estudantes (Martins, 2018).

Todo estudante da Universidade Federal do Ceará conhece a cena. É hora do almoço no Restaurante Universitário (RU). A fila se alonga pelo pátio, colegas conversam, alguns reclamam da espera, outros aproveitam o tempo para revisar o conteúdo da próxima prova. O tempo parece correr de forma diferente quando estamos na fila. Para alguns, são apenas alguns minutos. Para outros, parece uma eternidade.

Agora, pense um pouco. Quanto tempo, em média, um aluno passa esperando para se servir? Qual a chance de alguém que chega por volta das 12h30 esperar mais de vinte minutos? Por que em alguns dias a fila anda rápido e em outros parece não ter fim?

Essas perguntas podem parecer simples, mas abrem caminho para um universo fascinante. Elas revelam como a **Probabilidade e a Estatística** estão presentes em situações que vivemos todos os dias. A fila do RU não é apenas um detalhe da rotina estudantil. Ela é um retrato de como os fenômenos aleatórios acontecem ao nosso redor. Cada chegada de estudante, cada tempo de atendimento, cada variação de um dia para o outro forma um sistema dinâmico que pode ser estudado e compreendido.

É esse o convite deste livro. Explorar como traduzir a realidade em modelos, como usar simulações para observar padrões e como a Estatística pode ajudar a responder perguntas sobre o nosso cotidiano. Mais do que fórmulas, ela é uma maneira de olhar o mundo e encontrar nele sentido.

Aprender Estatística é aprender a lidar com a incerteza. É descobrir que até na fila do almoço existe conhecimento escondido, esperando para ser revelado.

1.1 Motivação

A Estatística, além de lidar com modelos matemáticos rigorosos, também é permeada por situações em que a intuição humana falha de maneira sistemática. Um exemplo clássico é o **problema do aniversário**, que há décadas desperta curiosidade entre estudantes e pesquisadores.

Enunciado

Em uma sala com r pessoas, qual a probabilidade de que pelo menos duas delas compartilhem o mesmo aniversário?

Um resultado surpreendente é: com apenas **23 pessoas** em uma sala, a probabilidade de que haja pelo menos uma coincidência de aniversários já é **superior a 50%**. Esse resultado é tão interessante que pode ser uma porta de entrada natural para discutir a diferença entre **probabilidade teórica** e **evidência empírica obtida por simulação**.

Da teoria à simulação

Do ponto de vista teórico, a probabilidade de que todos os aniversários sejam distintos entre r pessoas é

$$\Pr(\text{todos distintos}) = \prod_{i=1}^{r-1} \frac{365-i}{365} = \left(1 - \frac{1}{365}\right) \left(1 - \frac{2}{365}\right) \cdots \left(1 - \frac{r-1}{365}\right).$$

Logo, a probabilidade de pelo menos uma coincidência é

$$p_r = 1 - \Pr(\text{todos distintos}).$$

Esse produto é conceitualmente claro, mas fica pouco manejável mentalmente para r moderados. É aqui que a **simulação computacional** pode entrar como aliada didática e científica.

Um atalho analítico útil

O produto acima admite uma **aproximação exponencial simples e acurada**, obtida tomando logaritmo e usando a expansão para argumentos pequenos:

$$\ln(1-x) = -x + o(x), \quad (x \rightarrow 0).$$

Aplicando ao produto,

$$\begin{aligned}\ln(1 - p_r) &= \sum_{i=1}^{r-1} \ln\left(1 - \frac{i}{365}\right) \\ &\approx - \sum_{i=1}^{r-1} \frac{i}{365} = - \frac{1 + 2 + \dots + (r-1)}{365} = - \frac{r(r-1)}{2 \cdot 365}.\end{aligned}$$

Exponenciando e isolando p_r , obtemos a aproximação

$$p_r \approx 1 - \exp\left\{-\frac{r(r-1)}{730}\right\}.$$

Essa fórmula tem três virtudes didáticas:

- 1) **Clareza:** exhibe explicitamente o papel do número de pares $\binom{r}{2}$.
- 2) **Rapidez:** permite cálculos aproximados para valores de r de interesse.
- 3) **Boas aproximações** já para r na casa das dezenas.

⚠ Exemplo Rápido

- Para **23** pessoas:

$$p_{23}^{(\text{aprox})} = 1 - \exp\left\{-\frac{23 \cdot 22}{730}\right\} = 1 - \exp\{-0.69315\} \approx 0.500,$$

alinhando-se ao resultado clássico de que **23** pessoas já superam 50% de chance de coincidência.

O papel da simulação

A simulação estatística permite reproduzir o experimento de forma empírica: sorteamos aleatoriamente dias de aniversário para os indivíduos e verificamos se há repetições. Repetindo o processo milhares de vezes, obtemos uma estimativa para a probabilidade de coincidência.

Por exemplo, em **R**:

```
r <- 23
birthdays <- sample(1:365, r, replace = TRUE)
any(duplicated(birthdays))
```



```
[1] FALSE
```

Ao repetir esse procedimento muitas vezes (por exemplo, 10.000 simulações), podemos estimar a proporção de conjuntos com coincidência. Pela Lei dos Grandes Números, essa estimativa converge para o valor teórico de aproximadamente 0,507 quando $r = 23$.

```
set.seed(123) #reprodutibilidade

r <- 23
B <- 10000

acertos <- 0
i <- 0

repeat {
  i <- i + 1
  bdays <- sample(1:365, r, replace = TRUE)
  acertos <- acertos + as.integer(any(duplicated(bdays)))
  if (i >= B) break
}

p_hat <- acertos / B
p_hat
```

```
[1] 0.5073
```

Atividade: Problema do Aniversário (22 jogadores)

Nesta motivação consideramos um exemplo discutido em Martins (2018) que é o conhecido e amplamente divulgado problema do aniversário (ver, por exemplo, Falk, 2014). Martins (2018) segue o exemplo de Matthews; Stones (1998), considerando duas equipes de futebol e, portanto, coincidências de aniversário entre 22 jogadores. Martins (2018) afirma que um resultado positivo importante dessa atividade é a discussão que surgirá naturalmente entre os estudantes, com o professor atuando como mediador. Além disso, os estudantes adoram jogos e a descoberta prática, e a simulação facilita o engajamento nessas atividades, ao mesmo tempo que ilustra resultados que podem ser não intuitivos, bem como teoria geral, como a **Lei dos Grandes Números**.

Agora iremos considerar o seguinte problema:

Problema

Em uma partida de futebol, qual é a probabilidade de que pelo menos dois dos 22 jogadores façam aniversário no mesmo dia?

Em um país chamado de país do futebol, o contexto é proposital: o futebol é popular e as probabilidades resultantes são contraintuitivas. Antes de qualquer cálculo, considere as hipóteses: (i) todos os 365 dias do ano são igualmente prováveis para qualquer aniversário; (ii) as datas de aniversário dos jogadores são independentes entre si.

Objetivos

- Estimar, via simulação, a probabilidade de coincidência de aniversários.
- Relacionar frequência relativa, Lei dos Grandes Números e variação amostral.
- Comparar o resultado exato e aproximado.

Hipóteses

- 365 dias equiprováveis, datas independentes, ignorar bissexto/gêmeos.

Materiais

- R (ou Posit Cloud), roteiro com comandos `sample()`, `table()`, `mean()`.

Exercícios

- 1) Determinar o menor número de pessoas que deve estar em uma sala para que se possa apostar, com mais de 50% de chance de ganhar, que entre elas existam pelo menos duas com o mesmo aniversário.
- 2) Determinar o menor número de outras pessoas que deve estar em uma sala com você para que se possa apostar, com mais de 50% de chance de ganhar, que pelo menos uma delas tenha o mesmo aniversário que o seu.

1.2 Uma Pequena Introdução ao R

O R é peça-chave em inúmeros trabalhos de pesquisa e análise de dados porque reúne, de forma prática, um conjunto amplo de técnicas estatísticas atuais, das mais básicas às mais sofisticadas, e facilita seu uso no dia a dia. Quem começa no R, porém, muitas vezes também está dando os primeiros passos em programação. Assim, além de aprender as ferramentas do R para seus objetivos, é preciso desenvolver a “cabeça” de programador. Essa fase inicial ajuda

a explicar a fama de que o R é “difícil”. Mesmo assim, com prática e uma boa orientação, ele se revela bem mais acessível do que parece.

Instalação

Este guia mostra como **baixar e instalar o R a partir do CRAN** (Comprehensive R Archive Network), com orientações específicas para **Windows**, **macOS** e **Linux**. Ao final, você testará a instalação e configurará um **espelho (mirror) brasileiro** para baixar pacotes mais rápido.

Nota

O que é o CRAN?

É a rede oficial de servidores que distribui o R e seus pacotes. Você pode usar o endereço inteligente <https://cloud.r-project.org> ou definir um espelho no Brasil (ex.: C3SL/UFPR).

Você pode simplesmente usar:

- **Cloud CRAN (recomendado):** <https://cloud.r-project.org> (redireciona para um espelho próximo).
- **Brasil (ex.: UFPR/C3SL):** <https://cran-r.c3sl.ufpr.br>

Mais adiante, mostraremos como fixar o mirror no R permanentemente.

Windows

1. Acesse a página **Download R for Windows** → **base** e baixe o instalador.
2. Execute o instalador e avance com as opções padrão (recomendado para iniciantes).
3. (Opcional) Se pretende **compilar pacotes a partir do código-fonte**, instale o **Rtools** compatível com a sua versão do R.

Dica

Rtools: após instalar, reinicie o R/RStudio. Em geral, o Rtools adiciona as ferramentas ao *PATH* automaticamente.

macOS

1. Acesse **R for macOS** no CRAN e baixe o arquivo `.pkg` da versão atual.
2. Abra o `.pkg` e conclua a instalação.
3. (Opcional) Para compilar pacotes, instale também as **Command Line Tools** do Xcode:

```
xcode-select --install
```

Linux

Ubuntu/Debian

Opção rápida (repositório da distribuição):

```
sudo apt update  
sudo apt install -y r-base
```

Para obter versões mais novas (repositório do CRAN), siga as instruções do CRAN para adicionar o repositório oficial e então:

```
sudo apt update  
sudo apt install -y r-base r-base-dev
```

Verificar a instalação

No terminal/Prompt:

```
R --version  
R
```

Dentro do R:

```
version
```

```

platform      x86_64-pc-linux-gnu
arch          x86_64
os            linux-gnu
system        x86_64, linux-gnu
status
major         4
minor         5.2
year          2025
month         10
day           31
svn rev       88974
language      R
version.string R version 4.5.2 (2025-10-31)
nickname      [Not] Part in a Rumble

```

Se o R abriu no console, a instalação está ok!

Teste rápido de pacotes

No R:

```

#install.packages("tidyverse") # teste de instalação/espelho
library(tidyverse)

```

```

-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become

```

```

tibble(x = 1:5, y = x^2)

```

```

# A tibble: 5 x 2
      x     y
  <int> <dbl>
1     1     1

```

2	2	4
3	3	9
4	4	16
5	5	25

Se o pacote instalou e carregou sem erros, está tudo certo.

Fixar um mirror brasileiro do CRAN

Defina o espelho apenas nesta sessão:

```
options(repos = c(CRAN = "https://cran-r.c3sl.ufpr.br"))
install.packages("ggplot2")
```

Para tornar **permanente**, adicione a linha abaixo ao seu arquivo ~/.Rprofile:

```
options(repos = c(CRAN = "https://cloud.r-project.org"))
# ou, se preferir, o espelho da UFPR:
# options(repos = c(CRAN = "https://cran-r.c3sl.ufpr.br"))
```

Como editar o ~/.Rprofile

- Linux/macOS:

```
echo 'options(repos = c(CRAN = "https://cloud.r-project.org"))' >> ~/.Rprofile
```

- Windows: o ~ normalmente aponta para C:\\Users\\SEU_USUARIO\\Documents.

Você pode criar/editar C:\\Users\\SEU_USUARIO\\Documents\\.Rprofile com um editor de texto.

(Opcional) IDE recomendada: RStudio

Após instalar o R, instale o RStudio Desktop (Posit) para um ambiente de desenvolvimento mais amigável:

- Criação/edição de scripts
- Gerenciamento de projetos
- Visualização de plots e help integrados

Dicas e solução de problemas

- **Permissões de administrador:** em ambientes corporativos, pode ser necessário pedir para TI instalar o R.
- **Firewall/Proxy:** se a instalação de pacotes falhar, verifique configurações de proxy e tente trocar o mirror.
- **Compatibilidade de versões:** ao compilar pacotes, garanta que as ferramentas (Rtools no Windows; CLT/Xcode no macOS) correspondam à sua versão do R.
- Atualização do R: ao atualizar o R, alguns pacotes precisarão ser reinstalados; use `install.packages()` novamente.
- **Testes mínimos:**

```
sessionInfo()
```

```
R version 4.5.2 (2025-10-31)
Platform: x86_64-pc-linux-gnu
Running under: Ubuntu 24.04.4 LTS
```

```
Matrix products: default
```

```
BLAS: /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3
```

```
LAPACK: /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenblas-p-r0.3.26.so; LAPACK version 3.11.0
```

```
locale:
```

```
[1] LC_CTYPE=pt_BR.UTF-8      LC_NUMERIC=C
[3] LC_TIME=pt_BR.UTF-8      LC_COLLATE=pt_BR.UTF-8
[5] LC_MONETARY=pt_BR.UTF-8  LC_MESSAGES=pt_BR.UTF-8
[7] LC_PAPER=pt_BR.UTF-8     LC_NAME=C
[9] LC_ADDRESS=C             LC_TELEPHONE=C
[11] LC_MEASUREMENT=pt_BR.UTF-8 LC_IDENTIFICATION=C
```

```
time zone: America/Fortaleza
```

```
tzcode source: system (glibc)
```

```
attached base packages:
```

```
[1] stats      graphics  grDevices  utils      datasets  methods    base
```

```
other attached packages:
```

```
[1] lubridate_1.9.5 forcats_1.0.1  stringr_1.6.0  dplyr_1.2.1
[5] purrr_1.2.2     readr_2.2.0    tidyr_1.3.2    tibble_3.3.1
[9] ggplot2_4.0.3   tidyverse_2.0.0
```

loaded via a namespace (and not attached):

```
[1] gtable_0.3.6      jsonlite_2.0.0    compiler_4.5.2    tidyselect_1.2.1
[5] dichromat_2.0-0.1 scales_1.4.0      yaml_2.3.12      fastmap_1.2.0
[9] R6_2.6.1          generics_0.1.4    knitr_1.51       pillar_1.11.1
[13] RColorBrewer_1.1-3 tzdb_0.5.0        rlang_1.2.0      stringi_1.8.7
[17] xfun_0.59         S7_0.2.2          otel_0.2.0       timechange_0.4.0
[21] cli_3.6.6         withr_3.0.3       magrittr_2.0.5    digest_0.6.39
[25] grid_4.5.2        rstudioapi_0.19.0 hms_1.1.4        lifecycle_1.0.5
[29] vctrs_0.7.3       evaluate_1.0.5    glue_1.8.1       farver_2.1.2
[33] rmarkdown_2.31    tools_4.5.2       pkgconfig_2.0.3   htmltools_0.5.9
```

```
capabilities() # checa recursos gráficos, etc.
```

jpeg	png	tiff	tcltk	X11	aqua
TRUE	TRUE	TRUE	TRUE	FALSE	FALSE
http/ftp	sockets	libxml	fifo	cledit	iconv
TRUE	TRUE	FALSE	TRUE	FALSE	TRUE
NLS	Rprof	profmem	cairo	ICU	long.double
TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
libcurl					
TRUE					

1.3 O que é o R

R é uma implementação moderna da linguagem S, voltada à computação estatística e à visualização de dados. Ele reúne, no mesmo lugar, um ambiente interativo para análise e criação de gráficos e uma linguagem de programação completa.

Nota

Em uma frase: R é um ambiente estatístico + uma linguagem, criada para trabalhar bem com dados, gráficos e métodos modernos.

Principais características

- **Interativo e interpretado**, com suporte a **JIT/bytecode** via pacote `compiler`.
- **Orientado a objetos** (S3, S4 e R6) e com forte base **funcional**.

- **Modelo “tudo é objeto”**: números, vetores, data frames, funções, ambientes e modelos ajustados.
- **Vetorização nativa** e operações matriciais eficientes.
- **Ecossistema de pacotes** amplo (CRAN, Bioconductor) para estatística e ciência de dados.
- **Extensível** com C/C++/Fortran e integração com Python, SQL e serviços externos.
- **Multiplataforma** (Windows, macOS, Linux) e foco em **reprodutibilidade** (scripts, Quarto/R Markdown).

Exemplo - “tudo é objeto”

```
x <- 1:5          # vetor (objeto)
media <- mean(x)  # função aplicada ao objeto

f <- function(z) z^2
classe_f <- class(f) # "function" - funções também são objetos

attr(x, "nota") <- "exemplo de atributo"
lista <- list(x = x, media = media, classe_f = classe_f)

str(lista)        # inspeciona a estrutura
```

```
List of 3
 $ x      : int [1:5] 1 2 3 4 5
 ..- attr(*, "nota")= chr "exemplo de atributo"
 $ media   : num 3
 $ classe_f: chr "function"
```

```
lista
```

```
$x
[1] 1 2 3 4 5
attr("nota")
[1] "exemplo de atributo"

$media
[1] 3
```

```
$classe_f  
[1] "function"
```

Dica

Dica: use `str(obj)` para entender a estrutura de qualquer objeto no R.
Outras funções úteis: `class()`, `attributes()`, `typeof()`, `methods(class = ...)`.

Aqui vai como mudar o *prompt* do R para R>, na sessão atual usando:

```
options(prompt = "R> ", continue = "+")
```

Para adicionar comentários no R, como você observou nos exemplos acima, basta usar #, como no exemplo abaixo:

```
#Este é um comentário!  
#Que o semestre seja leve para todos nós!
```

Uma sessão ativa do R sempre está “apontando” para um diretório de trabalho. Se você não informar um caminho completo ao salvar ou importar arquivos, o R usará esse diretório por padrão. Para descobrir onde ele está:

```
getwd()
```

```
[1] "/home/manoel/Documentos/Book_Computacional_UFC"
```

Se você deseja modificar o diretório de trabalho, isso pode ser feito de maneira simples:

```
# Windows  
setwd("C:/Users/SeuUsuario/Documents/meu-projeto")  
  
# macOS / Linux  
setwd("/Users/seuusuario/meu-projeto")
```

O R não vem com todos os pacotes adicionais que a comunidade desenvolve. Para usá-los, você precisa baixar e instalar a partir de um repositório, em geral, o CRAN.

```
# define um espelho (mirror) do CRAN para esta sessão
options(repos = c(CRAN = "https://cloud.r-project.org"))

# instala um ou mais pacotes
install.packages("tidyverse")
install.packages(c("ggplot2", "dplyr", "readr"))

# carrega para usar
library(tidyverse)
```

A instalação pode ser feita também facilmente usando o RStudio.

Pacotes da comunidade são atualizados com frequência para corrigir erros e acrescentar recursos. Periodicamente, vale checar se há novas versões para o que você já tem instalado. A partir do console do R, você pode verificar e atualizar com um único comando.

```
# defina um mirror (opcional, mas recomendável)
options(repos = c(CRAN = "https://cloud.r-project.org"))

# procura versões mais novas e atualiza automaticamente
update.packages(ask = FALSE, checkBuilt = TRUE)
```

- ask = FALSE atualiza sem perguntar pacote a pacote.
- checkBuilt = TRUE recompila pacotes se sua versão do R mudou.

Ver o que está desatualizado antes:

```
old.packages() # retorna uma tabela com pacotes que têm versão mais nova
```

	Package
arrow	"arrow"
bitops	"bitops"
cards	"cards"
cardx	"cardx"
caTools	"caTools"
coin	"coin"
colorspace	"colorspace"
COMPOissonReg	"COMPOissonReg"
Deriv	"Deriv"
devtools	"devtools"
dichromat	"dichromat"

diffobj	"diffobj"
distributions3	"distributions3"
doBy	"doBy"
duckdb	"duckdb"
duckspatial	"duckspatial"
emmeans	"emmeans"
extraDistr	"extraDistr"
factoextra	"factoextra"
FactoMineR	"FactoMineR"
flextable	"flextable"
fs	"fs"
future	"future"
gert	"gert"
ggfun	"ggfun"
ggpubr	"ggpubr"
ggsci	"ggsci"
gh	"gh"
hexbin	"hexbin"
httr2	"httr2"
kableExtra	"kableExtra"
lava	"lava"
litedown	"litedown"
lme4	"lme4"
loo	"loo"
multcomp	"multcomp"
multcompView	"multcompView"
mvtnorm	"mvtnorm"
NLP	"NLP"
officer	"officer"
padr	"padr"
pak	"pak"
permutations	"permutations"
pkgdown	"pkgdown"
plotly	"plotly"
Polychrome	"Polychrome"
progressr	"progressr"
PROJ	"PROJ"
Rcpp	"Rcpp"
RcppArmadillo	"RcppArmadillo"
RcppParallel	"RcppParallel"
relations	"relations"
reproj	"reproj"
rlang	"rlang"

rsconnect	"rsconnect"
rstantools	"rstantools"
rstatix	"rstatix"
sandwich	"sandwich"
sf	"sf"
slam	"slam"
sp	"sp"
stars	"stars"
tseries	"tseries"
urlchecker	"urlchecker"
wooldridge	"wooldridge"
xfun	"xfun"
zip	"zip"
zoo	"zoo"
askpass	"askpass"
backports	"backports"
base64enc	"base64enc"
bit	"bit"
bit64	"bit64"
blob	"blob"
broom	"broom"
bslib	"bslib"
cachem	"cachem"
callr	"callr"
cli	"cli"
clipr	"clipr"
colorspace	"colorspace"
commonmark	"commonmark"
cpp11	"cpp11"
crayon	"crayon"
curl	"curl"
data.table	"data.table"
DBI	"DBI"
dbplyr	"dbplyr"
digest	"digest"
dplyr	"dplyr"
dtplyr	"dtplyr"
ellipsis	"ellipsis"
evaluate	"evaluate"
fansi	"fansi"
farver	"farver"
fastmap	"fastmap"
fontawesome	"fontawesome"

forcats	"forcats"
fs	"fs"
gargle	"gargle"
generics	"generics"
ggplot2	"ggplot2"
glue	"glue"
googledrive	"googledrive"
googlesheets4	"googlesheets4"
gtable	"gtable"
haven	"haven"
highr	"highr"
hms	"hms"
htmltools	"htmltools"
httpuv	"httpuv"
httr	"httr"
isoband	"isoband"
jsonlite	"jsonlite"
knitr	"knitr"
later	"later"
lifecycle	"lifecycle"
lubridate	"lubridate"
magrittr	"magrittr"
mime	"mime"
munsell	"munsell"
openssl	"openssl"
pillar	"pillar"
pkgKitten	"pkgKitten"
processx	"processx"
promises	"promises"
ps	"ps"
purrr	"purrr"
R6	"R6"
ragg	"ragg"
rappdirs	"rappdirs"
Rcpp	"Rcpp"
readr	"readr"
readxl	"readxl"
reprex	"reprex"
rJava	"rJava"
rlang	"rlang"
rmarkdown	"rmarkdown"
rstudioapi	"rstudioapi"
rvest	"rvest"

sass	"sass"	
scales	"scales"	
selectr	"selectr"	
shiny	"shiny"	
sourcetools	"sourcetools"	
stringi	"stringi"	
stringr	"stringr"	
sys	"sys"	
systemfonts	"systemfonts"	
textshaping	"textshaping"	
tibble	"tibble"	
tidyr	"tidyr"	
tidyselect	"tidyselect"	
timechange	"timechange"	
tinytex	"tinytex"	
tzdb	"tzdb"	
utf8	"utf8"	
uuid	"uuid"	
vctrs	"vctrs"	
viridisLite	"viridisLite"	
vroom	"vroom"	
withr	"withr"	
xfun	"xfun"	
xml2	"xml2"	
xtable	"xtable"	
yaml	"yaml"	
cluster	"cluster"	
MASS	"MASS"	
Matrix	"Matrix"	
nlme	"nlme"	
spatial	"spatial"	
survival	"survival"	
	LibPath	Installed
arrow	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"24.0.0"
bitops	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.0-9"
cards	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.8.0"
cardx	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.3.3"
caTools	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.18.3"
coin	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.4-3"
colorspace	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"2.1-2"
COMPOissonReg	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.8.1"
Deriv	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"4.2.0"
devtools	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"2.4.5"

dichromat	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"2.0-0.1"
diffobj	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.3.6"
distributions3	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.2.3"
doBy	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"4.7.1"
duckdb	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.5.4.2"
duckspatial	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.1.2"
emmeans	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"2.0.3"
extraDistr	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.10.0.4"
factoextra	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"2.1.0"
FactoMineR	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"2.15"
flextable	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.9.12"
fs	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.6.6"
future	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.70.0"
gert	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"2.3.1"
ggfun	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.2.0"
ggpubr	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.6.3"
ggsci	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"5.1.0"
gh	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.6.0"
hexbin	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.28.5"
httr2	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.2.3"
kableExtra	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.4.0"
lava	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.9.1"
litedown	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.9"
lme4	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"2.0-1"
loo	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"2.10.0"
multcomp	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.4-30"
multcompView	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.1-11"
mvtnorm	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.4-1"
NLP	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.3-2"
officer	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.7.5"
padr	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.6.3"
pak	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.10.0"
permutations	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.1-6"
pkgdown	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"2.2.0"
plotly	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"4.12.0"
Polychrome	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.5.4"
progressr	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.19.0"
PROJ	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.6.0"
Rcpp	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.1.1-1.1"
RcppArmadillo	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"15.4.0-1"
RcppParallel	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"5.1.11-2"
relations	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.6-17"
reproj	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.7.0"

rlang	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.2.0"
rsconnect	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.10.0"
rstantools	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"2.6.0"
rstatix	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.7.3"
sandwich	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"3.1-1"
sf	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.1-1"
slam	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.1-55"
sp	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"2.2-1"
stars	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.7-2"
tseries	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.10-61"
urlchecker	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.0.1"
wooldridge	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.4-4"
xfun	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"0.59"
zip	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"3.0.0"
zoo	"/home/manoel/R/x86_64-pc-linux-gnu-library/4.5"	"1.8-15"
askpass	"/usr/lib/R/site-library"	"1.2.0"
backports	"/usr/lib/R/site-library"	"1.4.1"
base64enc	"/usr/lib/R/site-library"	"0.1-3"
bit	"/usr/lib/R/site-library"	"4.0.5"
bit64	"/usr/lib/R/site-library"	"4.0.5"
blob	"/usr/lib/R/site-library"	"1.2.4"
broom	"/usr/lib/R/site-library"	"1.0.5"
bslib	"/usr/lib/R/site-library"	"0.6.1"
cachem	"/usr/lib/R/site-library"	"1.0.8"
callr	"/usr/lib/R/site-library"	"3.7.5"
cli	"/usr/lib/R/site-library"	"3.6.2"
clipr	"/usr/lib/R/site-library"	"0.8.0"
colorspace	"/usr/lib/R/site-library"	"2.1-0"
commonmark	"/usr/lib/R/site-library"	"1.9.1"
cpp11	"/usr/lib/R/site-library"	"0.4.7"
crayon	"/usr/lib/R/site-library"	"1.5.2"
curl	"/usr/lib/R/site-library"	"5.2.0"
data.table	"/usr/lib/R/site-library"	"1.14.10"
DBI	"/usr/lib/R/site-library"	"1.2.2"
dbplyr	"/usr/lib/R/site-library"	"2.4.0"
digest	"/usr/lib/R/site-library"	"0.6.34"
dplyr	"/usr/lib/R/site-library"	"1.1.4"
dtplyr	"/usr/lib/R/site-library"	"1.3.1"
ellipsis	"/usr/lib/R/site-library"	"0.3.2"
evaluate	"/usr/lib/R/site-library"	"0.23"
fansi	"/usr/lib/R/site-library"	"1.0.5"
farver	"/usr/lib/R/site-library"	"2.1.1"
fastmap	"/usr/lib/R/site-library"	"1.1.1"

fontawesome	"/usr/lib/R/site-library"	"0.5.2"
forcats	"/usr/lib/R/site-library"	"1.0.0"
fs	"/usr/lib/R/site-library"	"1.6.3"
gargle	"/usr/lib/R/site-library"	"1.5.2"
generics	"/usr/lib/R/site-library"	"0.1.3"
ggplot2	"/usr/lib/R/site-library"	"3.4.4"
glue	"/usr/lib/R/site-library"	"1.7.0"
googledrive	"/usr/lib/R/site-library"	"2.1.1"
googlesheets4	"/usr/lib/R/site-library"	"1.1.1"
gtable	"/usr/lib/R/site-library"	"0.3.4"
haven	"/usr/lib/R/site-library"	"2.5.4"
highr	"/usr/lib/R/site-library"	"0.10"
hms	"/usr/lib/R/site-library"	"1.1.3"
htmltools	"/usr/lib/R/site-library"	"0.5.7"
httpuv	"/usr/lib/R/site-library"	"1.6.14"
httr	"/usr/lib/R/site-library"	"1.4.7"
isoband	"/usr/lib/R/site-library"	"0.2.7"
jsonlite	"/usr/lib/R/site-library"	"1.8.8"
knitr	"/usr/lib/R/site-library"	"1.45"
later	"/usr/lib/R/site-library"	"1.3.2"
lifecycle	"/usr/lib/R/site-library"	"1.0.4"
lubridate	"/usr/lib/R/site-library"	"1.9.3"
magrittr	"/usr/lib/R/site-library"	"2.0.3"
mime	"/usr/lib/R/site-library"	"0.12"
munsell	"/usr/lib/R/site-library"	"0.5.0"
openssl	"/usr/lib/R/site-library"	"2.1.1"
pillar	"/usr/lib/R/site-library"	"1.9.0"
pkgKitten	"/usr/lib/R/site-library"	"0.2.3"
processx	"/usr/lib/R/site-library"	"3.8.3"
promises	"/usr/lib/R/site-library"	"1.2.1"
ps	"/usr/lib/R/site-library"	"1.7.6"
purrr	"/usr/lib/R/site-library"	"1.0.2"
R6	"/usr/lib/R/site-library"	"2.5.1"
ragg	"/usr/lib/R/site-library"	"1.2.7"
rappdirs	"/usr/lib/R/site-library"	"0.3.3"
Rcpp	"/usr/lib/R/site-library"	"1.0.12"
readr	"/usr/lib/R/site-library"	"2.1.5"
readxl	"/usr/lib/R/site-library"	"1.4.3"
reprex	"/usr/lib/R/site-library"	"2.1.0"
rJava	"/usr/lib/R/site-library"	"1.0-11"
rlang	"/usr/lib/R/site-library"	"1.1.3"
rmarkdown	"/usr/lib/R/site-library"	"2.25"
rstudioapi	"/usr/lib/R/site-library"	"0.15.0"

rvest	"/usr/lib/R/site-library"	"1.0.3"	
sass	"/usr/lib/R/site-library"	"0.4.8"	
scales	"/usr/lib/R/site-library"	"1.3.0"	
selectr	"/usr/lib/R/site-library"	"0.4-2"	
shiny	"/usr/lib/R/site-library"	"1.8.0"	
sourcetools	"/usr/lib/R/site-library"	"0.1.7-1"	
stringi	"/usr/lib/R/site-library"	"1.8.3"	
stringr	"/usr/lib/R/site-library"	"1.5.1"	
sys	"/usr/lib/R/site-library"	"3.4.2"	
systemfonts	"/usr/lib/R/site-library"	"1.0.5"	
textshaping	"/usr/lib/R/site-library"	"0.3.7"	
tibble	"/usr/lib/R/site-library"	"3.2.1"	
tidyr	"/usr/lib/R/site-library"	"1.3.1"	
tidyselect	"/usr/lib/R/site-library"	"1.2.0"	
timechange	"/usr/lib/R/site-library"	"0.3.0"	
tinytex	"/usr/lib/R/site-library"	"0.49"	
tzdb	"/usr/lib/R/site-library"	"0.4.0"	
utf8	"/usr/lib/R/site-library"	"1.2.4"	
uuid	"/usr/lib/R/site-library"	"1.2-0"	
vctrs	"/usr/lib/R/site-library"	"0.6.5"	
viridisLite	"/usr/lib/R/site-library"	"0.4.2"	
vroom	"/usr/lib/R/site-library"	"1.6.5"	
withr	"/usr/lib/R/site-library"	"2.5.0"	
xfun	"/usr/lib/R/site-library"	"0.41"	
xml2	"/usr/lib/R/site-library"	"1.3.6"	
xtable	"/usr/lib/R/site-library"	"1.8-4"	
yaml	"/usr/lib/R/site-library"	"2.3.8"	
cluster	"/usr/lib/R/library"	"2.1.8.2"	
MASS	"/usr/lib/R/library"	"7.3-65"	
Matrix	"/usr/lib/R/library"	"1.7-5"	
nlme	"/usr/lib/R/library"	"3.1-169"	
spatial	"/usr/lib/R/library"	"7.3-17"	
survival	"/usr/lib/R/library"	"3.8-6"	
	Built	ReposVer	Repository
arrow	"4.5.2"	"25.0.0"	"https://cloud.r-project.org/src/contrib"
bitops	"4.5.1"	"1.1-0"	"https://cloud.r-project.org/src/contrib"
cards	"4.5.2"	"0.8.1"	"https://cloud.r-project.org/src/contrib"
cardx	"4.5.2"	"0.3.4"	"https://cloud.r-project.org/src/contrib"
caTools	"4.5.1"	"1.18.4"	"https://cloud.r-project.org/src/contrib"
coin	"4.5.0"	"1.4-5"	"https://cloud.r-project.org/src/contrib"
colorspace	"4.5.1"	"2.1-3"	"https://cloud.r-project.org/src/contrib"
COMPOissonReg	"4.5.0"	"0.8.2"	"https://cloud.r-project.org/src/contrib"
Deriv	"4.5.1"	"4.3.0"	"https://cloud.r-project.org/src/contrib"

devtools	"4.5.1"	"2.5.2"	"https://cloud.r-project.org/src/contrib"
dichromat	"4.5.0"	"2.0-1"	"https://cloud.r-project.org/src/contrib"
diffobj	"4.5.1"	"0.3.8"	"https://cloud.r-project.org/src/contrib"
distributions3	"4.5.1"	"0.2.4"	"https://cloud.r-project.org/src/contrib"
doBy	"4.5.2"	"4.7.2"	"https://cloud.r-project.org/src/contrib"
duckdb	"4.5.2"	"1.5.5"	"https://cloud.r-project.org/src/contrib"
duckspatial	"4.5.2"	"1.2.1"	"https://cloud.r-project.org/src/contrib"
emmeans	"4.5.2"	"2.0.4"	"https://cloud.r-project.org/src/contrib"
extraDistr	"4.5.2"	"1.10.0.5"	"https://cloud.r-project.org/src/contrib"
factoextra	"4.5.2"	"2.2.0"	"https://cloud.r-project.org/src/contrib"
FactoMineR	"4.5.2"	"2.16"	"https://cloud.r-project.org/src/contrib"
flextable	"4.5.2"	"0.10.0"	"https://cloud.r-project.org/src/contrib"
fs	"4.5.1"	"2.1.0"	"https://cloud.r-project.org/src/contrib"
future	"4.5.2"	"1.75.0"	"https://cloud.r-project.org/src/contrib"
gert	"4.5.2"	"2.4.0"	"https://cloud.r-project.org/src/contrib"
ggfun	"4.5.1"	"0.2.1"	"https://cloud.r-project.org/src/contrib"
ggpubr	"4.5.2"	"1.0.0"	"https://cloud.r-project.org/src/contrib"
ggsci	"4.5.2"	"5.2.0"	"https://cloud.r-project.org/src/contrib"
gh	"4.5.2"	"1.6.1"	"https://cloud.r-project.org/src/contrib"
hexbin	"4.5.1"	"1.28.6"	"https://cloud.r-project.org/src/contrib"
httr2	"4.5.2"	"1.3.0"	"https://cloud.r-project.org/src/contrib"
kableExtra	"4.5.0"	"1.4.1"	"https://cloud.r-project.org/src/contrib"
lava	"4.5.2"	"1.9.2"	"https://cloud.r-project.org/src/contrib"
litedown	"4.5.2"	"0.10"	"https://cloud.r-project.org/src/contrib"
lme4	"4.5.2"	"2.0-6"	"https://cloud.r-project.org/src/contrib"
loo	"4.5.2"	"2.10.1"	"https://cloud.r-project.org/src/contrib"
multcomp	"4.5.2"	"1.4-31"	"https://cloud.r-project.org/src/contrib"
multcompView	"4.5.2"	"0.1-12"	"https://cloud.r-project.org/src/contrib"
mvtnorm	"4.5.2"	"1.4-2"	"https://cloud.r-project.org/src/contrib"
NLP	"4.5.0"	"0.3-3"	"https://cloud.r-project.org/src/contrib"
officer	"4.5.2"	"0.7.6"	"https://cloud.r-project.org/src/contrib"
padr	"4.5.2"	"0.7.0"	"https://cloud.r-project.org/src/contrib"
pak	"4.5.2"	"0.11.1"	"https://cloud.r-project.org/src/contrib"
permutations	"4.5.1"	"1.1-9-1"	"https://cloud.r-project.org/src/contrib"
pkgdown	"4.5.2"	"2.2.1"	"https://cloud.r-project.org/src/contrib"
plotly	"4.5.2"	"4.12.1"	"https://cloud.r-project.org/src/contrib"
Polychrome	"4.5.1"	"1.6.1"	"https://cloud.r-project.org/src/contrib"
progressr	"4.5.2"	"1.0.0"	"https://cloud.r-project.org/src/contrib"
PROJ	"4.5.1"	"0.7.0"	"https://cloud.r-project.org/src/contrib"
Rcpp	"4.5.2"	"1.1.2"	"https://cloud.r-project.org/src/contrib"
RcppArmadillo	"4.5.2"	"15.4.2-1"	"https://cloud.r-project.org/src/contrib"
RcppParallel	"4.5.2"	"6.2.0"	"https://cloud.r-project.org/src/contrib"
relations	"4.5.2"	"0.6-18"	"https://cloud.r-project.org/src/contrib"

reproj	"4.5.1"	"0.8.0"	"https://cloud.r-project.org/src/contrib"
rlang	"4.5.2"	"1.3.0"	"https://cloud.r-project.org/src/contrib"
rsconnect	"4.5.2"	"1.10.1"	"https://cloud.r-project.org/src/contrib"
rstantools	"4.5.2"	"2.7.0"	"https://cloud.r-project.org/src/contrib"
rstatix	"4.5.2"	"1.1.0"	"https://cloud.r-project.org/src/contrib"
sandwich	"4.5.0"	"3.1-2"	"https://cloud.r-project.org/src/contrib"
sf	"4.5.2"	"1.1-2"	"https://cloud.r-project.org/src/contrib"
slam	"4.5.0"	"0.1-56"	"https://cloud.r-project.org/src/contrib"
sp	"4.5.2"	"2.2-3"	"https://cloud.r-project.org/src/contrib"
stars	"4.5.2"	"0.7-3"	"https://cloud.r-project.org/src/contrib"
tseries	"4.5.2"	"0.10-62"	"https://cloud.r-project.org/src/contrib"
urlchecker	"4.5.1"	"2.0.0"	"https://cloud.r-project.org/src/contrib"
wooldridge	"4.5.0"	"1.4-7"	"https://cloud.r-project.org/src/contrib"
xfun	"4.5.2"	"0.60"	"https://cloud.r-project.org/src/contrib"
zip	"4.5.2"	"3.0.1"	"https://cloud.r-project.org/src/contrib"
zoo	"4.5.2"	"1.9-0"	"https://cloud.r-project.org/src/contrib"
askpass	"4.3.1"	"1.2.1"	"https://cloud.r-project.org/src/contrib"
backports	"4.1.2"	"1.5.1"	"https://cloud.r-project.org/src/contrib"
base64enc	"4.0.2"	"0.1-6"	"https://cloud.r-project.org/src/contrib"
bit	"4.2.2"	"4.6.0"	"https://cloud.r-project.org/src/contrib"
bit64	"4.0.2"	"4.8.2"	"https://cloud.r-project.org/src/contrib"
blob	"4.3.0"	"1.3.0"	"https://cloud.r-project.org/src/contrib"
broom	"4.3.0"	"1.0.13"	"https://cloud.r-project.org/src/contrib"
bslib	"4.3.2"	"0.11.0"	"https://cloud.r-project.org/src/contrib"
cachem	"4.3.0"	"1.1.0"	"https://cloud.r-project.org/src/contrib"
callr	"4.3.3"	"3.8.0"	"https://cloud.r-project.org/src/contrib"
cli	"4.3.2"	"3.6.6"	"https://cloud.r-project.org/src/contrib"
clipr	"4.1.2"	"0.8.1"	"https://cloud.r-project.org/src/contrib"
colorspace	"4.2.2"	"2.1-3"	"https://cloud.r-project.org/src/contrib"
commonmark	"4.3.2"	"2.0.0"	"https://cloud.r-project.org/src/contrib"
cpp11	"4.3.2"	"0.5.5"	"https://cloud.r-project.org/src/contrib"
crayon	"4.2.2"	"1.5.3"	"https://cloud.r-project.org/src/contrib"
curl	"4.3.3"	"7.1.0"	"https://cloud.r-project.org/src/contrib"
data.table	"4.3.2"	"1.18.4"	"https://cloud.r-project.org/src/contrib"
DBI	"4.3.2"	"1.3.0"	"https://cloud.r-project.org/src/contrib"
dbplyr	"4.3.2"	"2.6.0"	"https://cloud.r-project.org/src/contrib"
digest	"4.3.2"	"0.6.39"	"https://cloud.r-project.org/src/contrib"
dplyr	"4.3.2"	"1.2.1"	"https://cloud.r-project.org/src/contrib"
dtplyr	"4.3.0"	"1.3.3"	"https://cloud.r-project.org/src/contrib"
ellipsis	"4.1.1"	"0.3.3"	"https://cloud.r-project.org/src/contrib"
evaluate	"4.3.2"	"1.0.5"	"https://cloud.r-project.org/src/contrib"
fansi	"4.3.2"	"1.0.7"	"https://cloud.r-project.org/src/contrib"
farver	"4.2.1"	"2.1.2"	"https://cloud.r-project.org/src/contrib"

fastmap	"4.2.2"	"1.2.0"	"https://cloud.r-project.org/src/contrib"
fontawesome	"4.3.1"	"0.5.3"	"https://cloud.r-project.org/src/contrib"
forcats	"4.2.2"	"1.0.1"	"https://cloud.r-project.org/src/contrib"
fs	"4.3.3"	"2.1.0"	"https://cloud.r-project.org/src/contrib"
gargle	"4.3.1"	"1.6.1"	"https://cloud.r-project.org/src/contrib"
generics	"4.2.1"	"0.1.4"	"https://cloud.r-project.org/src/contrib"
ggplot2	"4.3.1"	"4.0.3"	"https://cloud.r-project.org/src/contrib"
glue	"4.3.2"	"1.8.1"	"https://cloud.r-project.org/src/contrib"
googledrive	"4.3.1"	"2.1.2"	"https://cloud.r-project.org/src/contrib"
googlesheets4	"4.3.1"	"1.1.2"	"https://cloud.r-project.org/src/contrib"
gtable	"4.3.1"	"0.3.6"	"https://cloud.r-project.org/src/contrib"
haven	"4.3.2"	"2.5.5"	"https://cloud.r-project.org/src/contrib"
highr	"4.2.2"	"0.12"	"https://cloud.r-project.org/src/contrib"
hms	"4.3.1"	"1.1.4"	"https://cloud.r-project.org/src/contrib"
htmltools	"4.3.2"	"0.5.9"	"https://cloud.r-project.org/src/contrib"
httpuv	"4.3.3"	"1.6.17"	"https://cloud.r-project.org/src/contrib"
httr	"4.3.1"	"1.4.8"	"https://cloud.r-project.org/src/contrib"
isoband	"4.2.2"	"0.3.0"	"https://cloud.r-project.org/src/contrib"
jsonlite	"4.3.2"	"2.0.0"	"https://cloud.r-project.org/src/contrib"
knitr	"4.3.2"	"1.51"	"https://cloud.r-project.org/src/contrib"
later	"4.3.2"	"1.4.8"	"https://cloud.r-project.org/src/contrib"
lifecycle	"4.3.2"	"1.0.5"	"https://cloud.r-project.org/src/contrib"
lubridate	"4.3.1"	"1.9.5"	"https://cloud.r-project.org/src/contrib"
magrittr	"4.1.2"	"2.0.5"	"https://cloud.r-project.org/src/contrib"
mime	"4.1.1"	"0.13"	"https://cloud.r-project.org/src/contrib"
munSELL	"4.0.1"	"0.5.1"	"https://cloud.r-project.org/src/contrib"
openssl	"4.3.3"	"2.4.2"	"https://cloud.r-project.org/src/contrib"
pillar	"4.3.0"	"1.11.1"	"https://cloud.r-project.org/src/contrib"
pkgKitten	"4.2.2"	"0.2.4"	"https://cloud.r-project.org/src/contrib"
processx	"4.3.2"	"3.9.0"	"https://cloud.r-project.org/src/contrib"
promises	"4.3.1"	"1.5.0"	"https://cloud.r-project.org/src/contrib"
ps	"4.3.2"	"1.9.3"	"https://cloud.r-project.org/src/contrib"
purrr	"4.3.1"	"1.2.2"	"https://cloud.r-project.org/src/contrib"
R6	"4.1.1"	"2.6.1"	"https://cloud.r-project.org/src/contrib"
ragg	"4.3.3"	"1.5.2"	"https://cloud.r-project.org/src/contrib"
rappdirs	"4.0.3"	"0.3.4"	"https://cloud.r-project.org/src/contrib"
Rcpp	"4.3.2"	"1.1.2"	"https://cloud.r-project.org/src/contrib"
readr	"4.3.2"	"2.2.0"	"https://cloud.r-project.org/src/contrib"
readxl	"4.3.1"	"1.5.0"	"https://cloud.r-project.org/src/contrib"
reprex	"4.3.2"	"2.1.1"	"https://cloud.r-project.org/src/contrib"
rJava	"4.3.2"	"1.0-18"	"https://cloud.r-project.org/src/contrib"
rlang	"4.3.2"	"1.3.0"	"https://cloud.r-project.org/src/contrib"
rmarkdown	"4.3.2"	"2.31"	"https://cloud.r-project.org/src/contrib"

rstudioapi	"4.3.1"	"0.19.0"	"https://cloud.r-project.org/src/contrib"
rvest	"4.2.1"	"1.0.5"	"https://cloud.r-project.org/src/contrib"
sass	"4.3.2"	"0.4.10"	"https://cloud.r-project.org/src/contrib"
scales	"4.3.2"	"1.4.0"	"https://cloud.r-project.org/src/contrib"
selectr	"4.0.1"	"0.6-0"	"https://cloud.r-project.org/src/contrib"
shiny	"4.3.2"	"1.14.0"	"https://cloud.r-project.org/src/contrib"
sourcetools	"4.2.2"	"0.1.7-2"	"https://cloud.r-project.org/src/contrib"
stringi	"4.3.2"	"1.8.7"	"https://cloud.r-project.org/src/contrib"
stringr	"4.3.2"	"1.6.0"	"https://cloud.r-project.org/src/contrib"
sys	"4.3.1"	"3.4.3"	"https://cloud.r-project.org/src/contrib"
systemfonts	"4.3.1"	"1.3.2"	"https://cloud.r-project.org/src/contrib"
textshaping	"4.3.1"	"1.0.5"	"https://cloud.r-project.org/src/contrib"
tibble	"4.3.1"	"3.3.1"	"https://cloud.r-project.org/src/contrib"
tidyr	"4.3.2"	"1.3.2"	"https://cloud.r-project.org/src/contrib"
tidyselect	"4.2.2"	"1.2.1"	"https://cloud.r-project.org/src/contrib"
timechange	"4.3.2"	"0.4.0"	"https://cloud.r-project.org/src/contrib"
tinytex	"4.3.2"	"0.60"	"https://cloud.r-project.org/src/contrib"
tzdb	"4.3.1"	"0.5.0"	"https://cloud.r-project.org/src/contrib"
utf8	"4.3.1"	"1.2.6"	"https://cloud.r-project.org/src/contrib"
uuid	"4.3.2"	"1.2-2"	"https://cloud.r-project.org/src/contrib"
vctrs	"4.3.2"	"0.7.3"	"https://cloud.r-project.org/src/contrib"
viridisLite	"4.3.1"	"0.4.3"	"https://cloud.r-project.org/src/contrib"
vroom	"4.3.2"	"1.7.1"	"https://cloud.r-project.org/src/contrib"
withr	"4.1.2"	"3.0.3"	"https://cloud.r-project.org/src/contrib"
xfun	"4.3.2"	"0.60"	"https://cloud.r-project.org/src/contrib"
xml2	"4.3.2"	"1.6.0"	"https://cloud.r-project.org/src/contrib"
xtable	"4.0.1"	"1.8-8"	"https://cloud.r-project.org/src/contrib"
yaml	"4.3.2"	"2.3.12"	"https://cloud.r-project.org/src/contrib"
cluster	"4.5.2"	"2.1.8.3"	"https://cloud.r-project.org/src/contrib"
MASS	"4.4.3"	"7.3-66"	"https://cloud.r-project.org/src/contrib"
Matrix	"4.5.3"	"1.7-6"	"https://cloud.r-project.org/src/contrib"
nlme	"4.5.3"	"3.1-170"	"https://cloud.r-project.org/src/contrib"
spatial	"4.4.0"	"7.3-18"	"https://cloud.r-project.org/src/contrib"
survival	"4.5.2"	"3.8-9"	"https://cloud.r-project.org/src/contrib"

Atualizar apenas alguns pacotes:

```
update.packages(oldPkgs = c("gamlss", "ggmap"), ask = FALSE)
```

O R traz um conjunto amplo de arquivos de ajuda que permitem:

- Buscar funcionalidades por nome ou por tema.

- Entender como usar uma função e quais argumentos ela recebe.
- Esclarecer o papel de cada argumento e o que a função retorna.
- Ver exemplos de uso prontos para rodar.
- Saber como citar o R, pacotes e conjuntos de dados em publicações.

Agora iremos ver algumas funções úteis:

- Ajuda com função específica:

```
?lm # atalho

help("lm") # abre a documentação da função

help(package = "stats") # índice de um pacote
```

- Buscar por tema (se você não sabe o nome exato):

```
??"linear model"      # busca por tópicos
help.search("normal") # alternativa
apropos("plot")        # lista objetos cujo nome contém "plot"
```

- Ver argumentos e formas de chamada

```
args(lm) # nomes dos argumentos (se aplicável)
formals(lm) # valores-padrão dos argumentos
```

- Exemplos, retorno e estrutura

```
example(lm) # roda os exemplos da ajuda
m <- lm(mpg ~ wt, data = mtcars)
str(m)      # inspeciona o objeto retornado
methods("predict") # métodos disponíveis para um genérico
```

- Vignettes (tutoriais)

```
vignette() # lista geral
vignette(package = "ggplot2") # vignettes de um pacote
browseVignettes(package = "dplyr") # abre no navegador
```

- Como citar R/pacotes/dados


```
citation() # como citar o R
```

To cite R in publications use:

```
R Core Team (2025). _R: A Language and Environment for Statistical  
Computing_. R Foundation for Statistical Computing, Vienna, Austria.  
<https://www.R-project.org/>.
```

Uma entrada BibTeX para usuários(as) de LaTeX é

```
@Manual{,  
  title = {R: A Language and Environment for Statistical Computing},  
  author = {{R Core Team}},  
  organization = {R Foundation for Statistical Computing},  
  address = {Vienna, Austria},  
  year = {2025},  
  url = {https://www.R-project.org/},  
}
```

We have invested a lot of time and effort in creating R, please cite it when using it for data analysis. See also 'citation("pkgname")' for citing R packages.

```
citation("ggplot2") # como citar um pacote
```

To cite ggplot2 in publications, please use

```
H. Wickham. ggplot2: Elegant Graphics for Data Analysis.  
Springer-Verlag New York, 2016.
```

Uma entrada BibTeX para usuários(as) de LaTeX é

```
@Book{,  
  author = {Hadley Wickham},  
  title = {ggplot2: Elegant Graphics for Data Analysis},  
  publisher = {Springer-Verlag New York},  
  year = {2016},  
  isbn = {978-3-319-24277-4},  
  url = {https://ggplot2.tidyverse.org/},  
}
```

```
toBibtex(citation("ggplot2")) # em BibTeX
```

```
@Book{,
  author = {Hadley Wickham},
  title = {ggplot2: Elegant Graphics for Data Analysis},
  publisher = {Springer-Verlag New York},
  year = {2016},
  isbn = {978-3-319-24277-4},
  url = {https://ggplot2.tidyverse.org},
}
```

- Conjuntos de dados

```
data() # abre uma lista com os datasets disponíveis

# ?mtcars # ajuda de um dataset específico
```

Console vs. editor: Você pode digitar comandos diretamente no console ou escrever um script no editor e então executá-lo. Em materiais didáticos, distinguimos os dois formatos para evitar confusão.

- 1) Código mostrado com prompt (R>) = digitado no console

- Em livros/notas, comandos de console costumam aparecer com o prompt na frente. Exemplo (divisão de 14 por 6):

```
R> options(prompt = "R> ", continue = "+ ")
R> 14/6
```

- Ao copiar para o R, remova o R>, o console já mostra o seu próprio prompt.

- 2) Código pensado para script (sem prompt)

- Quando o texto disser “escreva no editor e depois execute”, o código aparece sem R>. Exemplo (laço simples):

```
for (myitem in 5:7) {
  cat("--INÍCIO DO BLOCO--\n")
  cat("o item atual é", myitem, "\n")
  cat("--FIM DO BLOCO--\n\n")
}
```

```
--INÍCIO DO BLOCO--  
o item atual é 5  
--FIM DO BLOCO--
```

```
--INÍCIO DO BLOCO--  
o item atual é 6  
--FIM DO BLOCO--
```

```
--INÍCIO DO BLOCO--  
o item atual é 7  
--FIM DO BLOCO--
```

Dica

Dica de estilo: chaves na mesma linha do for, quebra de linha para o corpo e indentação consistente.

3) Linhas longas: uma linha só ou “quebradas”

- Chamadas extensas podem ficar em uma linha ou ser quebradas em pontos naturais (geralmente após vírgulas ou antes de argumentos nomeados).
- Ao quebrar, alinhe com o parêntese de abertura ou indente um nível.

Uma linha:

```
ordfac.vec <- factor(x = c("Small","Large","Large","Regular","Small"),  
                    levels = c("Small","Regular","Large"),  
                    ordered = TRUE)
```

Quebrada (equivalente, apenas formatação):

```
ordfac.vec2 <-  
  factor(  
    x      = c("Small","Large","Large","Regular","Small"),  
    levels = c("Small","Regular","Large"),  
    ordered = TRUE  
  )  
identical(ordfac.vec, ordfac.vec2)
```

```
[1] TRUE
```

1.4 Números, Aritmética, Atribuição e Vetores

1) Números (*numerics*)

Em R, o tipo numérico padrão é **double** (ponto flutuante). Você também pode ter **inteiros** (sufixo L) e **complexos**.

```
typeof(1)      # "double"
```

```
[1] "double"
```

```
typeof(1L)     # "integer"
```

```
[1] "integer"
```

```
typeof(1+2i)   # "complex"
```

```
[1] "complex"
```

- Valores especiais:

```
1/0 # Inf
```

```
[1] Inf
```

```
-1/0 # -Inf
```

```
[1] -Inf
```

```
0/0 # NaN
```

```
[1] NaN
```

```
is.finite(c(Inf, 3.14)) # checa finitude
```

```
[1] FALSE TRUE
```

💡 Dica

Precisão numérica: comparações com `==` podem falhar por arredondamento binário. Prefira `all.equal(x, y)`.

```
# 1) Exemplo clássico
x <- 0.1 + 0.2
y <- 0.3

x == y                                # pode dar FALSE
```

```
[1] FALSE
```

```
identical(x, y)                       # compara "bit a bit": quase sempre FALSE aqui
```

```
[1] FALSE
```

```
# Ver a diferença real (use mais dígitos)
old <- options(digits = 17)
x; y; x - y                           # diferença minúscula de arredondamento binário
```

```
[1] 0.30000000000000004
```

```
[1] 0.29999999999999999
```

```
[1] 5.5511151231257827e-17
```

```
options(old)

# Comparação com tolerância
x;y
```

```
[1] 0.3
```

```
[1] 0.3
```

```
all.equal(x, y) # Se "quase iguais", retorna TRUE. Se diferem, retorna uma string
```

```
[1] TRUE
```

```
isTRUE(all.equal(x, y)) # retorna um lógico (TRUE/FALSE)
```

```
[1] TRUE
```

```
# 2) Outros casos que sofrem com ponto flutuante  
sin(pi) # ~ 1.224646799e-16 (não é 0 exato)
```

```
[1] 1.224647e-16
```

```
sin(pi) == 0 # FALSE
```

```
[1] FALSE
```

```
all.equal(sin(pi), 0) # TRUE
```

```
[1] TRUE
```

```
# 3) Ajustando a tolerância manualmente (quando precisar ser mais/menos rígido)  
all.equal(x, y, tolerance = 1e-12) # ainda TRUE
```

```
[1] TRUE
```

```
all.equal(x, y, tolerance = 1e-20) # agora acusa diferença
```

```
[1] "Mean relative difference: 1.850372e-16"
```

```
# 4) Estratégia base sem all.equal: comparar o |erro| com um limite  
abs((sqrt(2))^2 - 2) < 1e-12 # TRUE (aceita "igualdade" numérica com folga)
```

```
[1] TRUE
```

2) Aritmética

Operadores básicos: + - * / ^ e, ainda, divisão inteira %/% e módulo %%.

```
14/6
```

```
[1] 2.333333
```

```
14 %/% 6 # quociente inteiro
```

```
[1] 2
```

```
14 %% 6 # resto
```

```
[1] 2
```

```
2^3 # potência
```

```
[1] 8
```

- Precedência: $\wedge \rightarrow * / \%/\% \% \rightarrow + -$. Use **parênteses para deixar claro!**

3) Atribuição

O padrão recomendado é `<-`. `=` também atribui, mas use-o preferencialmente para argumentos de função. `->` é atribuição para a direita. `<<-` afeta o ambiente pai (use com parcimônia).

```
x <- 10
y = 5    # ok, mas prefira <- fora de chamadas de função
z <- x + y

z -> resultado # atribuição para a direita (menos comum)
resultado
```

```
[1] 15
```

```
# Exemplo de <<- (evite, pode dificultar depuração)
contador <- local({
  n <- 0
  function() { n <<- n + 1; n }
})
contador(); contador(); contador()
```

```
[1] 1
```

```
[1] 2
```

```
[1] 3
```

 Aviso

Boas práticas: Evite <- sempre que possível; prefira retornar valores e trabalhar com escopos explícitos.

4) Vetores (atômicos)

Vetores são coleções unidimensionais do mesmo tipo: lógico, inteiro, double, caractere, complexo ou raw.

```
v <- c(2, 4, 6, 8)
length(v); typeof(v)
```

```
[1] 4
```

```
[1] "double"
```

```
seq(1, 5, by = 2)      # sequência
```

```
[1] 1 3 5
```

```
1:5                    # atalho
```

```
[1] 1 2 3 4 5
```

```
rep(3, times = 4)      # repetição
```

```
[1] 3 3 3 3
```

- *Coerção automática:* ao misturar tipos, R promove para um tipo comum.


```
c(1, TRUE, "a") # tudo vira character
```

```
[1] "1"      "TRUE" "a"
```

Aviso

Quando você faz `c(1, TRUE, "a")`, o R precisa escolher um tipo comum para todos os elementos. A regra de coerção (promoção de tipos) segue, a grosso modo:

logical integer double complex character

- Indexação e filtragem:

```
x <- c(10, 20, 30, 40, 50)
x[3]          # posição
```

```
[1] 30
```

```
x[-1]         # tudo, exceto o 1º
```

```
[1] 20 30 40 50
```

```
x[x > 25]     # filtragem lógica
```

```
[1] 30 40 50
```

```
which(x > 25)  # posições TRUE
```

```
[1] 3 4 5
```

```
names(x) <- letters[1:5]
x["c"]
```

```
c
30
```

4.1. Vetorização e reciclagem

A maioria das operações é vetorizada (aplica-se elemento a elemento). Quando os comprimentos diferem, R tenta reciclar o menor vetor.

```
a <- 1:5
b <- 10
a + b # soma escalar-vetor (b é reciclado)
```

```
[1] 11 12 13 14 15
```

```
a + c(1, 2) # reciclagem com aviso (5 não é múltiplo de 2)
```

Warning in `a + c(1, 2)`: comprimento do objeto maior não é múltiplo do comprimento do objeto menor

```
[1] 2 4 4 6 6
```

4.2. Nomes × Objetos e copy-on-modify

```
x <- 1:3
y <- x
y[1] <- 99
x # permanece 1 2 3
```

```
[1] 1 2 3
```

```
y # 99 2 3
```

```
[1] 99 2 3
```

Aviso

Exceção: ambientes e estruturas por referência (p.ex., R6) não seguem copy-on-modify.

Exercícios

1. Crie um vetor com os números de 5 a 15 e calcule a média.
2. Use `seq()` para gerar 0, 0.5, 1, ..., 5.
3. Mostre com código a diferença entre `%/%` e `%%` usando 14 e 6.
4. Atribua `x <- 1:4` e some com o vetor `c(10, 20)`. O que acontece?

5. Verifique o tipo de 1, 1L e 1+0i. Explique a diferença entre tipo (typeof) e classe (class).
6. Crie e armazene uma sequência de valores de 5 a -11, progredindo em passos de 0.3.
7. Reescreva o objeto de (6) com a mesma sequência invertida.
8. Repita o vetor c(-1, 3, -5, 7, -9) duas vezes, com cada elemento repetido 10 vezes. Apresente os dados ordenados do maior para o menor.
9. Crie e armazene um vetor contendo, em qualquer ordem:
 - a) inteiros de 6 a 12 (inclusive);
 - b) 5.3 repetido 3 vezes;
 - c) o número -3;
 - d) Uma sequência de nove valores começando em 102 e terminando no número que é o comprimento total do vetor criado em (8).
10. Confirme que o comprimento do vetor criado em (9) é 20.

1.5 Matrizes

A matriz é uma construção matemática importante e é essencial para muitos métodos estatísticos. Normalmente descreve-se uma matriz A como uma matriz $m \times n$; isto é, A terá exatamente m linhas e n colunas. Isso significa que A terá um total de mn entradas, com cada entrada a i, j ocupando uma posição única dada por sua linha específica ($i = 1, 2, \dots, m$) e coluna ($j = 1, 2, \dots, n$).

Para criar uma matriz no R, use o comando, apropriadamente chamado, `matrix`, fornecendo as entradas da matriz ao argumento `data` como um vetor:

```
A <- matrix(
  data = c(-3, 2, 893, 0.17),
  nrow = 2, #linha
  ncol = 2  #colunas
)
```

A

```
      [,1] [,2]
[1,]  -3 893.00
[2,]   2  0.17
```

É importante estar ciente de como o R preenche a matriz usando as entradas de `data`. Observando o exemplo anterior, você pode ver que a matriz *A* foi preenchida coluna por coluna ao ler as entradas de dados da esquerda para a direita. Você pode controlar como o R preenche os dados usando o argumento `byrow`, como mostrado nos exemplos a seguir:

```
matrix(data = c(1,2,3,4,5,6), nrow = 2, ncol = 3, byrow = FALSE)
```

```
      [,1] [,2] [,3]
[1,]     1     3     5
[2,]     2     4     6
```

Qual a diferença na construção da matriz ao usar `byrow = TRUE`?

Também é possível construir matrizes usando os comandos `cbind` e `rbind`. Veja os exemplos abaixo:

```
cbind(c(1,4), c(2,5), c(3,6))
```

```
      [,1] [,2] [,3]
[1,]     1     2     3
[2,]     4     5     6
```

```
rbind(1:3,4:6)
```

```
      [,1] [,2] [,3]
[1,]     1     2     3
[2,]     4     5     6
```

Para saber a dimensão, número de linhas e colunas de uma matriz você pode usar os seguintes comandos:

```
dim(A); nrow(A); ncol(A)
```

```
[1] 2 2
```

```
[1] 2
```

```
[1] 2
```

Considere a seguinte matriz:

```
A <- matrix(c(0.3, 4.5, 55.3, 91, 0.1, 105.5, -4.2, 8.2, 27.9), nrow = 3, ncol = 3)
```

A

```
      [,1] [,2] [,3]
[1,]  0.3  91.0 -4.2
[2,]  4.5   0.1  8.2
[3,] 55.3 105.5 27.9
```

Para dizer ao R para “olhar para a terceira linha de A e me dar o elemento da segunda coluna”, você executa o seguinte:

```
A[3, 2]
```

```
[1] 105.5
```

Como esperado, você recebe o elemento na posição $[3, 2]$.

Você também pode identificar os valores ao longo da diagonal de uma matriz quadrada (isto é, uma matriz com igual número de linhas e colunas) usando o comando `diag`.

```
diag(A)
```

```
[1]  0.3  0.1 27.9
```

Exercícios

(a) Construa e armazene uma matriz 4×2 preenchida **por linhas** com os valores: 4.3, 3.1, 8.2, 8.2, 3.2, 0.9, 1.6 e 6.5 (nessa ordem).

```
A <- matrix(c(4.3, 3.1, 8.2, 8.2, 3.2, 0.9, 1.6, 6.5), nrow = 4, byrow = TRUE); A
```

```
      [,1] [,2]
[1,]  4.3  3.1
[2,]  8.2  8.2
[3,]  3.2  0.9
[4,]  1.6  6.5
```

(b) Confirme que as dimensões da matriz do item (a) são 3×2 se você remover qualquer uma das linhas.

```
matrix_test_32 <- function(data){  
  
  v_saida <- NULL  
  for(i in 1:nrow(data)){  
  
    v_saida[i] <- all(dim(data[-i, , drop = FALSE]) == c(3,2))  
  }  
  
  sprintf( "A matriz é 3x2? %s", ifelse(all(v_saida) == TRUE, "Sim", "Não" ) )  
}  
  
matrix_test_32(A)
```

```
[1] "A matriz é 3x2? Sim"
```

```
B <- matrix(c(1,2,3,4), ncol = 2); B
```

```
      [,1] [,2]  
[1,]     1     3  
[2,]     2     4
```

```
matrix_test_32(B)
```

```
[1] "A matriz é 3x2? Não"
```

(c) Sobrescreva a **segunda coluna** da matriz do item (a) com essa mesma coluna **ordenada do menor para o maior**.

(d) O que o R retorna se você excluir a **quarta linha** e a **primeira coluna** do item (c)? Use **matrix** para garantir que o resultado seja uma **matriz de uma única coluna**, e não um vetor.

(e) Armazene os **quatro elementos inferiores** do item (c) como uma nova matriz 2×2 .

(f) Sobrescreva, **nesta ordem**, os elementos do item (c) nas posições (4, 2), (1, 2), (4, 1) e (1, 1) com $-\frac{1}{2}$ dos dois valores na **diagonal do item (e)**.

1.6 Operações com matrizes

Em R, a transposta de uma matriz se obtém com `t()`, ela troca linhas por colunas.

```
A <- rbind(c(2,5,2),c(6,1,4))  
t(A)
```

	[,1]	[,2]
[1,]	2	6
[2,]	5	1
[3,]	2	4

Você pode criar uma matriz identidade de qualquer dimensão usando a função `matrix`, mas há uma forma mais rápida usando `diag`. Antes, usei `diag` em uma matriz existente para extrair ou sobrescrever seus elementos da diagonal. Você também pode usá-la assim:

```
I4 <- diag(4) # 4x4  
I4
```

	[,1]	[,2]	[,3]	[,4]
[1,]	1	0	0	0
[2,]	0	1	0	0
[3,]	0	0	1	0
[4,]	0	0	0	1

```
I5 <- diag(1, nrow = 5, ncol = 5) #forma explicita  
I5
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	1	0	0	0	0
[2,]	0	1	0	0	0
[3,]	0	0	1	0	0
[4,]	0	0	0	1	0
[5,]	0	0	0	0	1

O R realizará essa multiplicação de maneira elemento a elemento, como você poderia esperar. A multiplicação escalar de uma matriz é realizada usando o operador aritmético padrão `*`.

```
A <- rbind(c(2,5,2) , c(6,1,4))
a <- 2

a*A
```

```
      [,1] [,2] [,3]
[1,]    4   10    4
[2,]   12    2    8
```

Você pode somar ou subtrair quaisquer duas matrizes de mesmo tamanho usando os símbolos padrão + e -.

```
A <- cbind(c(2,5,2), c(6,1,4))

A
```

```
      [,1] [,2]
[1,]    2    6
[2,]    5    1
[3,]    2    4
```

```
B <- cbind(c(-2,3,6), c(8.1,8.2,-9.8))

B
```

```
      [,1] [,2]
[1,]   -2  8.1
[2,]    3  8.2
[3,]    6 -9.8
```

```
A - B
```

```
      [,1] [,2]
[1,]    4 -2.1
[2,]    2 -7.2
[3,]   -4 13.8
```

```
A + B
```



```

      [,1] [,2]
[1,]    0 14.1
[2,]    8  9.2
[3,]    8 -5.8

```

Ao contrário da adição, subtração e multiplicação escalar, a multiplicação de matrizes não é um cálculo elemento a elemento, e o operador padrão `*` não pode ser usado. Em vez disso, você deve usar o operador de produto matricial do R, escrito com símbolos de porcentagem como `%*%`. Antes de tentar esse operador, vamos primeiro armazenar as duas matrizes de exemplo e verificar se o número de colunas na primeira matriz corresponde ao número de linhas na segunda matriz usando `dim`.

```

A <- rbind(c(2,5,2), c(6,1,4))
dim(A)

```

```

[1] 2 3

```

```

B <- cbind(c(3,-1,1), c(-3,1,5))
dim(B)

```

```

[1] 3 2

```

Isso confirma que as duas matrizes são compatíveis para a multiplicação, então você pode prosseguir.

```

A%*%B

```

```

      [,1] [,2]
[1,]    3    9
[2,]   21    3

```

Você pode mostrar que a multiplicação de matrizes é **não comutativa** usando as mesmas duas matrizes. Inverter a ordem da multiplicação produz um resultado completamente diferente.

```

B%*%A

```

```

      [,1] [,2] [,3]
[1,]  -12  12  -6
[2,]   4  -4   2
[3,]  32  10  22

```

Matrizes que não são invertíveis são chamadas de singulares. Inverter uma matriz é frequentemente necessário ao resolver sistemas de equações e tem implicações práticas importantes. Há vários métodos para inversão, e o custo computacional cresce muito à medida que o tamanho da matriz aumenta. Não entraremos em detalhes aqui; para discussões formais, veja Golub e Van Loan (1989). Por ora, veja a função `solve` do R como uma opção para inverter matrizes.

```

A <- matrix(data = c(3, 4, 1, 2), nrow = 2, ncol = 2)

A

```

```

      [,1] [,2]
[1,]    3    1
[2,]    4    2

```

```

solve(A)

```

```

      [,1] [,2]
[1,]    1 -0.5
[2,]   -2  1.5

```

Você também pode verificar que o produto dessas duas matrizes (usando as regras de multiplicação de matrizes) resulta na matriz identidade 2×2

```

A %*% solve(A)

```

```

      [,1] [,2]
[1,]    1    0
[2,]    0    1

```

Exercícios

(a) Calcule

$$\frac{2}{7} \left(\begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 7 & 6 \end{bmatrix} - \begin{bmatrix} 10 & 20 \\ 30 & 40 \\ 50 & 60 \end{bmatrix} \right).$$

(b) Armazene as duas matrizes abaixo:

$$A = \begin{bmatrix} 1 \\ 2 \\ 7 \end{bmatrix}, \quad B = \begin{bmatrix} 3 \\ 4 \\ 8 \end{bmatrix}.$$

Quais das multiplicações a seguir são possíveis? Para as que forem, calcule o resultado.

- i. $A \cdot B$;
- ii. $A^\top \cdot B$;
- iii. $B^\top \cdot (A \cdot A^\top)$;
- iv. $(A \cdot A^\top) \cdot B^\top$;
- v. $[(B \cdot B^\top) + (A \cdot A^\top) - 100 I_3]^{-1}$.

(c) Para

$$A = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 5 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix},$$

confirme que $A^{-1} \cdot A - I_4$ fornece uma matriz 4×4 de zeros.

1.7 Laços e repetições

O laço clássico, ao estilo Fortran, está disponível no R. A sintaxe é um pouco diferente, mas a ideia é idêntica: você pede que um índice, i , assuma uma sequência de valores, e que uma ou mais linhas de comandos sejam executadas tantas vezes quantos forem os valores distintos de i . A seguir, um laço executado cinco vezes com i de 1 a 5; imprimimos o quadrado de cada valor:

```
for(i in 1:5) print(i^2)
```

```
[1] 1  
[1] 4  
[1] 9  
[1] 16  
[1] 25
```

Para várias linhas de código, você usa chaves `{}` para incluir o material sobre o qual o laço deve atuar. Note que a “quebra de linha” (pressionar a tecla Enter) ao final de cada comando é uma parte essencial da estrutura (você pode substituir as quebras de linha por ponto e vírgula, se preferir, mas a clareza melhora se cada comando for colocado em uma linha separada).

```
j <- k <- 0  
for(i in 1:5){  
  j <- j + 1  
  k <- k + i*j  
  print(i + j + k)  
}
```

```
[1] 3  
[1] 9  
[1] 20  
[1] 38  
[1] 65
```

O fatorial de um número inteiro x (escrito $x!$) é o produto de todos os inteiros positivos de 1 até x . Por definição:

$$x! = x \times (x - 1) \times (x - 2) \times (x - 3) \cdots \times 2 \times 1.$$

Por exemplo, $4! = 4 \times 3 \times 2 \times 1 = 24$. Aqui está a função:

```
fac1 <- function(x){
  f <- 1
  if(x < 2) return(1)
  for(i in 2:x){
    f <- f*i
  }
  f
}
```

Parece complicado para uma tarefa simples. Mas vamos testar de 0 a 5:

```
sapply(0:5, fac1)
```

```
[1] 1 1 2 6 24 120
```

Existem outras duas funções de laço no R: **repeat** e **while**. Vamos demonstrar seu uso para fins de ilustração, mas podemos fazer bem melhor ao escrever uma função compacta para calcular fatoriais (veja abaixo). Primeiro, a função **while**:

```
fac2 <- function(x){
  f <- 1
  t <- x
  while(t > 1){
    f <- f*t
    t <- t - 1
  }
  return(f)
}
```

Agora iremos testar a função para os números inteiros de 0 a 5:

```
sapply(0:5, fac2)
```

```
[1] 1 1 2 6 24 120
```

O ponto central

o **while** sozinho não gera a sequência; você precisa configurar e atualizar manualmente uma variável de controle (aqui **t**). Isso torna o **while** menos compacto do que **for**, mas

útil quando você não sabe de antemão quantas iterações serão necessárias.

Por fim, vamos mostrar o uso da função `repeat`:

```
fac3 <- function(x){  
  f <- 1  
  t <- x  
  repeat{  
    if(t<2) break  
    f <- f*t  
    t <- t - 1  
  }  
  return(f)  
}
```

O `repeat` em R não tem um fim explícito, ele vai rodar para sempre a menos que você coloque uma condição de saída que use `break`.

```
sapply(0:5,fac3)
```

```
[1] 1 1 2 6 24 120
```

É uma boa prática de programação em R evitar o uso de laços sempre que possível. O uso de funções vetorizadas torna isso particularmente simples em muitos casos. Suponha que você queira substituir todos os valores negativos de um array por zeros. Você poderia ter escrito algo assim:

```
for(i in 1:length(y)){  
  if(y[i] < 0) y[i] <- 0  
}
```

No entanto, você pode simplesmente fazer assim:

```
y[y<0] <- 0
```

Às vezes você quer fazer uma coisa se uma condição for verdadeira e outra diferente se a condição for falsa (em vez de não fazer nada, como no exemplo anterior). A função `ifelse` permite fazer isso em vetores inteiros sem precisar usar laços com `for`. Por exemplo, podemos querer substituir qualquer valor negativo de `y` por `-1` e qualquer valor positivo ou zero por `+1`:

```
y <- c(-5, -1, 0, 2, 7)
ifelse(y < 0, -1, 1)
```

```
[1] -1 -1  1  1  1
```

1.8 Escrevendo funções no R

Normalmente você escreve funções em R para realizar operações que exigem duas ou mais linhas de código para serem executadas, e que você não quer digitar várias vezes. Podemos querer escrever funções simples para calcular medidas de tendência central, calcular fatoriais e coisas do tipo.

Funções em R são objetos que realizam operações sobre argumentos que lhes são fornecidos e retornam um ou mais valores. A sintaxe para escrever uma função é

```
function(lista_de_argumentos){
  # corpo da função
  # instruções
  return(valor)
}
```

O primeiro componente da declaração de função é a palavra-chave **function**, que indica ao R que você quer criar uma função.

Uma **lista de argumentos** é uma lista separada por vírgulas de **argumentos formais**. Um argumento formal pode ser:

- um símbolo (isto é, um nome de variável como **x** ou **y**),
- uma declaração do tipo **símbolo = expressão** (por exemplo, **pch = 16**),
- ou o argumento especial **...** (três pontos), que permite passar múltiplos argumentos adicionais.

O corpo pode ser qualquer expressão válida em R ou um conjunto de expressões em uma ou mais linhas. Em geral, o corpo é um grupo de expressões dentro de chaves **{ }**, com cada expressão em uma linha separada (se o corpo couber em uma linha, as chaves não são necessárias).

Funções normalmente são atribuídas a símbolos (variáveis), mas isso não é obrigatório. Esse conceito só começa a fazer sentido depois de ver vários exemplos em funcionamento.

1.8.1 Média Aritmética

A média é a soma dos valores y_i dividida pela quantidade de valores n (somando ao longo do número de elementos do vetor y). Em R, $n = \text{length}(y)$ e $\sum y = \text{sum}(y)$. Assim, uma função para calcular a média aritmética é:

```
media_aritmetica <- function(x) sum(x)/length(x)
```

Vamos testar a função com dados em que sabemos de antemão qual deve ser o resultado.

```
y <- c(3, 3, 4, 5, 5)
media_aritmetica(y)
```

```
[1] 4
```

1.8.2 Mediana

A mediana (ou 50º percentil) é o valor do meio dos valores ordenados de um vetor de números:

```
sort(y)[ceiling(length(y)/2)]
```

Há, é claro, um pequeno problema aqui, porque se o vetor contém um número par de números, então não existe valor do meio. A lógica aqui é que precisamos calcular a média aritmética dos dois valores de y em cada lado do meio. Surge agora a questão de como sabemos, em geral, se o vetor contém um número ímpar ou par de números, de modo que possamos decidir qual dos dois métodos usar. O truque aqui é usar o módulo 2. Agora temos todas as ferramentas de que precisamos para escrever uma função geral para calcular medianas. Vamos chamar a função de `mediana` e defini-la assim:

```
mediana <- function(x){
  impar_par <- length(x)%%2
  if(impar_par == 0) (sort(x)[length(x)/2] + sort(x)[1 + length(x)/2])/2
  else sort(x)[ceiling(length(x)/2)]
}
```

Vamos agora testar a função:

```
mediana(y)
```

```
[1] 4
```


Exercícios

1. Crie sua própria função que calcula a variância, o desvio-padrão e o coeficiente de variação.
2. Crie uma função para listar os elementos de uma série de Fibonacci (1, 1, 2, 3, 5, 8, ...).
3. Crie uma função para calcular fatoriais usando o comando `cumprod`.
4. Crie sua própria função para calcular a mediana.
5. Crie sua própria função para calcular a média.

2 Geração de Números Pseudoaleatórios

As simulações estatísticas requerem uma base inicial formada por números que imitem a aleatoriedade. Formalmente, buscamos uma sequência $U_1, U_2, \dots$ de variáveis independentes e identicamente distribuídas (iid) seguindo a distribuição uniforme no intervalo $(0, 1)$.

2.1 Geração de sequências $U(0, 1)$

Uma abordagem é utilizar dispositivos físicos aleatorizadores, como máquinas que sorteiam números de loteria, roletas ou circuitos eletrônicos que produzem “ruído aleatório”. Contudo, tais dispositivos apresentam desvantagens:

1. **Baixa velocidade** e dificuldade de integração direta com computadores.
2. **Necessidade de reprodutibilidade** da sequência. Por exemplo, para verificação de código ou comparação de políticas em um modelo de simulação, usando a mesma sequência para reduzir a variância da diferença entre resultados.

Uma forma simples de obter reprodutibilidade é armazenar a sequência em um dispositivo de memória (HD, CD-ROM, livro). De fato, a RAND Corporation publicou *A Million Random Digits with 100 000 Random Normal Deviates* (1955). Entretanto, acessar armazenamento externo milhares ou milhões de vezes torna a simulação lenta.

Nota

- Também existem fontes para números aleatórios reais na Internet.
- O www.random.org oferece números aleatórios verdadeiros para qualquer pessoa na internet. A aleatoriedade vem do ruído atmosférico, que para muitos propósitos é melhor do que os algoritmos de números pseudoaleatórios normalmente usados em programas de computador. As pessoas usam os números para operar loterias, sorteios e apostas, e para seus jogos e sites de apostas.
- Em www.randomnumbers.info é oferecida a possibilidade de baixar números aleatórios gerados usando um gerador quântico de números aleatórios sob demanda.

2.2 Por que “Pseudoaleatórios”?

Diferente de dispositivos físicos (como ruído atmosférico ou decaimento quântico), que geram aleatoriedade real mas são lentos e irreprodutíveis, os computadores utilizam algoritmos determinísticos. Esses números são chamados de pseudoaleatórios porque, embora pareçam aleatórios sob testes estatísticos, são gerados por fórmulas matemáticas fixas.

A abordagem computacional é preferida por três razões fundamentais:

1. *Eficiência*: Geração extremamente rápida em tempo de execução.
2. *Economia de Memória*: Elimina a necessidade de armazenar tabelas imensas de números.
3. *Reprodutibilidade*: Através do uso de uma semente (**seed**), é possível replicar exatamente a mesma sequência, o que é vital para depuração de código e redução de variância em comparações de modelos.

Entretanto, a escolha inadequada da recorrência pode gerar sequências com baixa qualidade estatística. Um dos métodos mais antigos e influentes para isso é o **Gerador Congruencial Linear (GCL)**.

2.3 O Gerador Congruencial Linear (GCL)

O GCL é um dos métodos mais clássicos para gerar sequências pseudoaleatórias. Ele produz uma sequência de inteiros x_n através da recorrência:

$$x_n = (a x_{n-1} + c) \bmod m$$

em que, $m > 0$ é o **módulo**, a é o **multiplicador**, c é o **incremento** e x_0 é a **semente** (*seed*), escolhida pelo usuário. O resultado final é obtido normalizando os valores:

$$u_n = \frac{x_n}{m}, \quad \text{com } u_n \in (0, 1).$$

2.3.1 Escolha de Parâmetros e Período

A qualidade do gerador depende criticamente da escolha de a , c e m . Como os valores de x_n estão no conjunto $\{0, 1, \dots, m-1\}$, a sequência inevitavelmente se repetirá após, no máximo, m sorteios. Esse intervalo antes da repetição é chamado de período.

Para maximizar o período e a qualidade estatística, seguem-se algumas diretrizes:

- Módulo (m): Deve ser um número primo grande, geralmente próximo à capacidade da palavra do computador (ex. $m = 2^{31} - 1$ para máquinas de 32 bits).
- Multiplicador (a): Para $m = 2^{31} - 1$, uma escolha comum e testada é $a = 7^5 = 16807$.
- Valores históricos:
 - **Park–Miller (1988)**: $m = 2^{31} - 1$, $a = 16807$, $c = 0$.
 - **Numerical Recipes (1992)**: $m = 2^{32}$, $a = 1664525$, $c = 1013904223$.

2.4 Implementação no R

No ambiente R, não precisamos implementar esses geradores manualmente para uso cotidiano, pois o *software* oferece funções otimizadas.

- `runif(n)`: Gera n valores uniformes (0,1).
- `set.seed(semente)`: Define o ponto de partida da sequência. Se não for definida, o R utiliza o relógio do sistema como semente padrão.

```
# Exemplo de reprodutibilidade
set.seed(123)
runif(5) # Gera 5 números
```

```
[1] 0.2875775 0.7883051 0.4089769 0.8830174 0.9404673
```

```
set.seed(123)
runif(5) # Gera EXATAMENTE os mesmos 5 números
```

```
[1] 0.2875775 0.7883051 0.4089769 0.8830174 0.9404673
```

Por que o GCL funciona?

1. **Simplicidade**: apenas uma multiplicação, uma soma e uma operação de módulo.
2. **Velocidade**: pode ser implementado de forma extremamente eficiente.
3. **Controle**: dependendo da escolha de a , c , m , é possível obter um período longo, ou seja, muitos números gerados antes de a sequência se repetir.

Exemplos

```
# Gerador Congruencial Linear em Python

def gcl(seed, a, c, m, n = 10):
    "Gera n números pseudoaleatórios normalizados em (0,1)."
```

$$x = \text{seed}$$

```
    seq = []
    for _ in range(n):
        x = (a * x + c) % m
        seq.append(x / m)
    return seq

# Exemplo: Park-Miller (multiplicativo)
m = 2**31 - 1
a = 16807
c = 0
seed = 12345

gcl(seed, a, c, m, n=10)
```

[0.09661652850760917, 0.8339946273872604, 0.9477024976851895, 0.035878594981449935, 0.011545

```
# Gerador Congruencial Linear em R

gcl <- function(seed, a, c, m, n=10) {
  x <- seed
  seq <- numeric(n)
  for (i in 1:n) {
    x <- (a * x + c) %% m
    seq[i] <- x / m
  }
  seq
}

# Exemplo: Park-Miller
m <- 2^31 - 1
a <- 16807
c <- 0
seed <- 12345
```

```
gcl(seed, a, c, m, n=10)
```

```
[1] 0.09661653 0.83399463 0.94770250 0.03587859 0.01154585 0.05115522  
[7] 0.76578717 0.58492974 0.91413005 0.78380039
```

2.5 Como validar um Gerador?

Um “bom” gerador deve passar por verificações rápidas para garantir que as propriedades i.i.d. sejam mantidas.

1. Histograma: Os valores devem se distribuir de forma plana (uniforme) entre 0 e 1.
2. Gráfico de Pares (U_i, U_{i+1}): Ajuda a detectar dependências seriais. Os pontos devem preencher o quadrado unitário sem padrões ou linhas visíveis (evitando o “agrupamento” ou *clumping*).
3. Autocorrelação: A correlação entre valores sucessivos deve ser desprezível.

Limitações

Apesar de sua importância histórica, os GCLs apresentam limitações:

- O período, mesmo que longo, é finito.
- Podem apresentar correlações indesejadas em dimensões mais altas.
- Não são recomendados para aplicações de alta segurança (como criptografia).

Hoje, geradores como o **Mersenne Twister** substituíram o GCL em muitas linguagens (por exemplo, é o padrão no R e no NumPy). Ainda assim, o GCL é fundamental para entender os princípios da geração de números pseudoaleatórios.

Exercícios

1. Implemente um GCL (multiplicativo ou misto) em Python ou R.
2. Gere 1000 números pseudoaleatórios e faça:
 - Um histograma para verificar se a distribuição se parece com a uniforme (0, 1).
 - Um gráfico de dispersão de pares consecutivos (u_n, u_{n+1}).
3. Compare o comportamento quando:

- Usa os parâmetros clássicos de Park–Miller ($m = 2^{31} - 1, a = 16807, c = 0$).
- Usa parâmetros pequenos, por exemplo $m = 17, a = 5, c = 1$.

Pergunta para reflexão: O que acontece com a qualidade dos números gerados quando usamos parâmetros pequenos?

i Park–Miller hoje: ainda vale a pena?

O gerador congruencial linear clássico de **Park–Miller** foi proposto em 1988, com parâmetros:

$$m = 2^{31} - 1 \approx 2,147,483,647, \quad a = 16807, \quad c = 0.$$

Na época, esses valores eram ideais para computadores de **32 bits**, embora a implementação precisasse de alguns cuidados para evitar **overflow**.

Nos computadores modernos de **64 bits**, esse problema desaparece: o produto $a \times x$ cabe confortavelmente nos registradores, e a implementação é direta e eficiente.

Vantagens atuais:

- Simples e rápido.
- Fácil de implementar em qualquer linguagem.
- Período máximo de mais de **2 bilhões** de números.

Limitações:

- Hoje, esse período pode ser considerado curto para simulações muito extensas.
- Geradores modernos, como o **Mersenne Twister** (período $2^{19937} - 1$) ou a família **GCP (Gerador Congruencial Permutado)**, oferecem propriedades estatísticas superiores.

Em resumo: O Park–Miller ainda é excelente para fins didáticos e pequenas simulações, mas para aplicações científicas de grande escala recomenda-se usar geradores mais robustos.

Existem no R vários algoritmos para geradores de números pseudoaleatórios. Para ver quais são, basta:

```
help(Random)
```

Random Number Generation

Description

`.Random.seed` is an integer vector, containing the random number generator (RNG) **state** for random number generation in **R**. It can be saved and restored, but should not be altered by the user.

`RNGkind` is a more friendly interface to query or set the kind of RNG in use.

`RNGversion` can be used to set the random generators as they were in an earlier **R** version (for reproducibility).

`set.seed` is the recommended way to specify seeds.

Usage

```
.Random.seed <- c(rng.kind, n1, n2, ...)  
  
RNGkind(kind = NULL, normal.kind = NULL, sample.kind = NULL)  
RNGversion(vstr)  
set.seed(seed, kind = NULL, normal.kind = NULL, sample.kind = NULL)
```

Arguments

kind character or `NULL`. If **kind** is a character string, set **R**'s RNG to the kind desired. Use "default" to return to the **R** default. See 'Details' for the interpretation of `NULL`.

Figura 2.1: Imagem da documentação.

3 Simulação de Variáveis Aleatórias

3.1 Caso Discreto

3.1.1 Método da Transformação Inversa

Suponha que desejamos gerar o valor de uma variável aleatória discreta X com função de massa de probabilidade (f.m.p):

$$\Pr(X = x_j) = p_j, \quad j = 0, 1, \dots, \quad \text{e} \quad \sum_j p_j = 1.$$

Para realizar isso, gere um valor u de $U \sim (0, 1)$ e atribua o valor x_j , $j = 0, 1, \dots$, a X conforme as condições abaixo:

$$X = \begin{cases} x_0, & \text{se } u < p_0, \\ x_1, & \text{se } p_0 \leq u < p_0 + p_1, \\ x_2, & \text{se } p_0 + p_1 \leq u < p_0 + p_1 + p_2, \\ x_3, & \text{se } p_0 + p_1 + p_2 \leq u < p_0 + p_1 + p_2 + p_3, \\ \vdots & \\ x_j, & \text{se } \sum_{i=0}^{j-1} p_i \leq u < \sum_{i=0}^j p_i, \\ \vdots & \end{cases}$$

Como, para $0 < a < b < 1$, $\Pr(a \leq U < b) = b - a$, temos que:

$$\Pr(X = x_j) = \Pr\left(\sum_{i=0}^{j-1} p_i \leq U < \sum_{i=0}^j p_i\right) = p_j,$$

e, portanto, X possui a distribuição desejada.

Observações:

1. Algoritmo - O procedimento anterior pode ser escrito como:

Passo 1. Gerar um valor $u \sim U(0, 1)$.

Passo 2. Se $u < p_0$, então $X = x_0$; caso contrário:

- se $u < p_0 + p_1$, então $X = x_1$; caso contrário:
- se $u < p_0 + p_1 + p_2$, então $X = x_2$; caso contrário:
- $\vdots$

Passo 3. Repetir os passos 1 e 2 n vezes, onde n é o tamanho da amostra.

2. Se os valores, $x_i, i \geq 0$, são ordenados de modo que $x_0 < x_1 < \dots$, e de denotamos

F como função de distribuição de X , então $F(x_j) = \sum_{i=1}^j p_i$. Assim,

X será igual a x_j se $F(x_{j-1}) \leq U < F(x_j)$.

Em outras palavras, após gerar um número aleatório U , determinamos o valor de X encontrando o intervalo $[F(x_{j-1}), F(x_j))$ no qual U se encontra (ou, de forma equivalente, encontrando o inverso de $F(U)$). É por essa razão que o procedimento acima é denominado *método da transformada inversa discreta* para gerar X .

i Nota

O tempo necessário para gerar uma variável aleatória discreta por esse método é proporcional ao número de intervalos que devem ser pesquisados. Por essa razão, às vezes é vantajoso considerar os possíveis valores x_j de X em ordem decrescente das probabilidades p_j .

Exemplo 1

Simular n valores de X tal que $p_1 = 0.20$, $p_2 = 0.15$, $p_3 = 0.25$, $p_4 = 0.40$, em que $p_j = P(X = j)$.

Algoritmo 1

Passo 1. Gerar n valores $u \sim U(0, 1)$.

Passo 2. Para cada u :

- Se $u < 0.20$, então $X = 1$.
- Caso contrário, se $u < 0.35$, então $X = 2$.

- Caso contrário, se $u < 0.60$, então $X = 3$.
- Caso contrário ($u \geq 0.60$), então $X = 4$.

Passo 3. Repetir n vezes os passos 1 e 2.

A seguir, são apresentados dos códigos escritos na linguagem R:

```
# Proposta 1
n <- 10000
x <- 1:4
p <- c(0.20, 0.15, 0.25, 0.40)
pa <- cumsum(p) # probabilidades acumuladas
a <- c() # vetor para a amostra gerada de X

for(i in 1:n){
  u <- runif(1)
  if (u < pa[1]) {
    a[i] <- x[1]
  } else {
    if (u < pa[2]) {
      a[i] <- x[2]
    } else { if (u < pa[3]) {
      a[i] <- x[3]
    } else {
      a[i] <- x[4]
    }
  }
}
}
table(a)/n # tabela de proporções
```

```
a
      1      2      3      4
0.2009 0.1533 0.2531 0.3927
```

```
# Proposta 2
n <- 10000;
x <- 1:4;
p <- c(0.20, 0.15, 0.25, 0.40)
pa <- cumsum(p) # probabilidades acumuladas
a <- c() # vetor para a amostra gerada de X
```

```

for(i in 1:n){
  u = runif(1)
  ifelse(u < pa[1], a[i] <- x[1],
    ifelse(u < pa[2], a[i] <- x[2],
      ifelse(u < pa[3], a[i] <- x[3], a[i] <- x[4])))
}
table(a)/n # tabela de proporções

```

```

a
      1      2      3      4
0.1987 0.1498 0.2475 0.4040

```

Algoritmo 2

Passo 1. Gerar um valor $u \sim U(0, 1)$.

Passo 2. Para cada u :

- Se $u < 0.40$, então $X = 4$;
- Caso contrário, se $u < 0.65$, então $X = 3$;
- Caso contrário, se $u < 0.85$, então $X = 1$;
- Caso contrário $u \geq 0.85$, então $X = 2$.

Passo 3. Repetir n vezes os passos 1 e 2.

Observação:

A proposta 2 usa a **ordem decrescente** dos p_j para otimizar a busca.

Algoritmo 3 (mudando as desigualdades)

Passo 1. Gerar $u \sim U(0, 1)$.

Passo 2. Se $u \leq 0.40$, então $X = 4$;

- senão, se $u \leq 0.65$, então $X = 3$;
- senão, se $u \leq 0.85$, então $X = 1$;
- senão ($u > 0.85$), então $X = 2$.

Passo 3. Repetir n vezes os passos 1–2.

Observação:

Com $u \sim U(0,1)$ é contínua, usar $<$ ou $\leq$ é equivalente em termos de distribuição (a probabilidade de u cair exatamente no ponto de corte é zero). Em implementação numérica, essa escolha apenas define para onde vão raríssimos empates nos pontos cortes.

Se você deseja simular valores de X com distribuição uniforme discreta:

$$\Pr(X = j) = 1/k, \quad j = 1, 2, 3, \dots, k.$$

Basta utilizar a Proposições 1 ou a Proposição 2.

Proposição 1:

Gere $U \sim U(0,1)$. Defina $X = j$ se

$$\frac{j-1}{k} \leq U < \frac{j}{k},$$

ou, equivalentemente,

$$(j-1) \leq kU < j.$$

Proposição 2:

Gere $U \sim U(0,1)$. Defina $X = \lfloor kU \rfloor + 1$, em que $\lfloor z \rfloor$ é o maior inteiro menor ou igual a z .

Exemplo 2

Para simular valores de X com $\Pr(X = j) = 1/10$, $j = 1, 2, 3, \dots, 10$, você pode seguir o procedimento a seguir:

U	kU	$X = \lfloor kU \rfloor + 1$
0,01	0,1	1
0,31	3,1	4
0,53	5,3	6
0,92	9,2	10
0,45	4,5	5
$\vdots$	$\vdots$	$\vdots$
0,74	7,4	8

Exercícios

1. Gerar uma permutação dos números $1, 2, 3, \dots, n$, considerando todas as $n!$ possíveis permutações igualmente prováveis.
2. Gerar valores de $X \sim \text{Geométrica}(p)$.
3. Implementar o algoritmo para gerar valores de $X \sim \text{Poisson}(\lambda)$.
4. Implementar o algoritmo para gerar valores de $X \sim \text{Binomial}(n, p)$.

Exemplo 3

Nosso objetivo agora é gerar valores de $X \sim \text{Poisson}(\lambda)$ com:

$$p_i = \Pr(X = i) = \frac{e^{-\lambda} \lambda^i}{i!}, \quad i = 0, 1, 2, \dots$$

Identidade importante:

$$p_{i+1} = \frac{\lambda}{i+1} p_i, \quad i \geq 0.$$

Uma forma bastante utilizada para gerar valores de uma variável aleatória $X \sim \text{Poisson}(\lambda)$ é por meio de algoritmos de simulação baseados na identidade acima.

Algoritmo 4

Passo 1. Gerar um número aleatório $u \sim U(0, 1)$.

Passo 2. Inicializar $i = 0$, $p = e^{-\lambda}$, $F = p$.

Passo 3. Se $u < F$, então definir $X = i$ e parar.

Passo 4. Caso contrário:

- atualizar $i \leftarrow i + 1$,
- atualizar $p \leftarrow \frac{\lambda}{i} p$,
- atualizar $F \leftarrow F + p$,
- voltar ao passo 3.

Exercício

Complete o código R abaixo:

```
N = 10^5 # tamanho da amostra
L = 3 # lambda
x = c()
for (j in 1:N){
  u = runif(1)
  i = 0; p = exp(-L); F = p
  aceito = "não"
  while (aceito != "sim"){
    if(u < F) {
      ...
    } else {
      ...
    }
  }
}
```

Exemplo 4

Por fim, iremos gerar valores da variável aleatório X , $X \sim \text{Binomial}(n, p)$, com f.m.p dada por:

$$\Pr(X = i) = \frac{n!}{i!(n-i)!} p^i (1-p)^{n-i}, \quad i = 0, 1, 2, \dots, n.$$

! Identidade importante:

$$\Pr(X = i + 1) = \frac{n-i}{i+1} \frac{p}{1-p} \Pr(X = i).$$

A partir do resultado acima e do método da transformação inversa podemos escrever o seguinte algoritmo:

Algoritmo 5

Passo 1. Gerar um número aleatório $u \sim U(0, 1)$.

Passo 2. Calcular $k = \frac{p}{1-p}$, inicializar $i = 0$, $p_r = (1-p)^n$, $F = p_r$.

Passo 3. Se $u < F$, definir $X = i$ e parar.

Passo 4. Caso contrário:

- atualizar $p_r \leftarrow \frac{n-i}{i+1} k p_r$,
- atualizar $F \leftarrow F + p_r$,
- atualizar $i \leftarrow i + 1$,
- voltar ao passo 3.

Exercício 5

Escreva um código na linguagem R baseado no Algoritmo 5. Utilize alguma ferramenta gráfica para verificar a coerência dos resultados.

3.1.2 Método da Aceitação-Rejeição

Suponha que haja interesse em simular valores de uma v.a. X e que não seja possível inverter a sua função de distribuição ou não dispomos de um método para gerar dessa variável aleatória. Entretanto, sabemos como simular de uma outra v.a. Y e que é possível estabelecer uma relação entre as probabilidades associadas às duas variáveis aleatórias (X e Y) de tal modo que valores de Y possam ser admitidos como valores de X .

x	1	2	3	4	5	6	7	8	9	10
p_x	0,11	0,12	0,09	0,08	0,12	0,10	0,09	0,09	0,10	0,10

y	1	2	3	4	5	6	7	8	9	10
q_y	0,10	0,10	0,10	0,10	0,10	0,10	0,10	0,10	0,10	0,10

Contexto:

- **Podemos:** simular valores de Y com f.p. $q_j = \Pr(Y = j), j \geq 0$.
- **Queremos:** simular valores de X com f.p. $p_j = \Pr(X = j), j \geq 0$.
- **Proposta:** simular valor y de Y e aceitar esse valor para X com probabilidade proporcional a $\frac{p_y}{q_y}$.

Algoritmo

Passo 1. Simular y de Y com f.m.p. q_y .
 Passo 2. Gerar $u \sim U(0, 1)$.

Passo 3. Se $u < \frac{p_y}{c q_y}$, então aceite $X = y$. Caso contrário, rejeite e não atribua valor a X .

Passo 4. Repetir os passos 1–3 até obter o tamanho de amostra desejado.

O algoritmo aceitação-rejeição gera uma v.a. X tal que $\Pr(X = j) = p_j$, $j = 0, 1, 2, \dots$. Além disso, o número de iterações que o algoritmo necessita para obter X é uma v.a. geométrica com média c .

Prova do Teorema

1. Em uma iteração, determinar a probabilidade de gerar e ser aceito o valor j :

$$\Pr(Y = j, \text{aceitar}) = \Pr(Y = j) \cdot \Pr(\text{aceitar} \mid Y = j) = q_j \cdot \frac{p_j}{c q_j} = \frac{p_j}{c}.$$

2. Calcular a probabilidade de aceitar um valor j gerado:

$$\Pr(\text{aceitar}) = \sum_j \Pr(Y = j, \text{aceitar}) = \sum_j \frac{p_j}{c} = \frac{1}{c}.$$

3. Como cada iteração independentemente resulta um valor aceitável com probabilidade $\frac{1}{c}$, o número de iterações necessárias segue uma geométrica de média c . Portanto,

$$\Pr(X = j) = \sum_n P(j \text{ aceito na iteração } n) = \sum_n \left(1 - \frac{1}{c}\right)^{n-1} \cdot \frac{p_j}{c} = p_j.$$

Observações

- A constante c está relacionada com o número de interações necessárias até a aceitação de um valor de Y para X .
- O valor c deve ser o menor possível.
- O valor de c será o $\max \left\{ \frac{p_y}{q_y} \right\}$.

Nota

$$u < \frac{p_y}{c \cdot q_y} \quad \text{e} \quad P(U < \frac{p_y}{c \cdot q_y}) = \frac{p_y}{c \cdot q_y}$$

$\Downarrow$

$$\frac{p_y}{c \cdot q_y} \leq 1 \quad \text{para todo } y \text{ tal que } p_y > 0$$

$$\Downarrow$$

$$c = \max \left\{ \frac{p_y}{q_y} \right\}$$

Exemplo 1

Gerar um valor da variável aleatória X com f.m.p.:

j	1	2	3	4	5	6	7	8	9	10
p_j	0,11	0,12	0,09	0,08	0,12	0,10	0,09	0,09	0,10	0,10

Considerando que sabemos gerar de uma v.a. uniforme discreta, assumiremos Y com distribuição

$$\Pr(Y = j) = q_j = \frac{1}{10}; \quad j = 1, 2, \dots, 10.$$

A constante c será determinada por $c = \max \left\{ \frac{p_j}{q_j} \right\} = \frac{0,12}{0,10} = 1,2$.

Algoritmo

1. Simular y de Y : gere $u_1 \sim U(0, 1)$ e faça $y = \text{Int}(10u_1) + 1$.
2. Gerar um segundo número aleatório u_2 .
3. Se $u_2 < \frac{p_y}{0,12}$, faça $X = y$ e pare. Caso contrário, retorne ao passo 1.

i Nota

Suponha $y = 1$. Então, se $u_2 < \frac{p_1}{0,12} = \frac{0,11}{0,12} = 0,9167$, faremos $X = 1$. Isto é, assumiremos que o valor 1 gerado é plausível de ser da distribuição de X .

```
set.seed(20252)
pseudo_x <- NULL
for(i in seq_len(100000)){
```

```

pj <- c(0.11, 0.12, 0.09, 0.08, 0.12, 0.10, 0.09, 0.09, 0.10, 0.10)

repeat{
  u1 <- runif(1)
  y <- floor(10*u1) + 1

  u2 <- runif(1)
  x <- y

  if(u2 < pj[y]/0.12) break
}

pseudo_x[i] <- x
}

round(prop.table(table(pseudo_x)),2)

```

```

pseudo_x
  1    2    3    4    5    6    7    8    9   10
0.11 0.12 0.09 0.08 0.12 0.10 0.09 0.09 0.10 0.10

```

Existe algum problema com os resultados acima? Se sim, faça a correção do código e verifique graficamente a coerência dos resultados.

Observações

- 91,67% de todos os valores 1 gerados de Y serão aceitos
- 100% dos valores 2 gerados de Y serão aceitos
- 75% dos valores 3 gerados de Y serão aceitos
- Qualquer valor da constante c inferior a 1,2 impossibilita gerar a distribuição de X
- Qualquer valor da constante c superior a 1,2 tornaria o processo mais lento para a obtenção da amostra

Exercícios

1. Gere números pseudoaleatórios de X considerando $c = 2, 4$.
2. Compare com os resultados obtidos no Exemplo.

3.1.3 Método da Composição

Suponha que tenhamos um método eficiente para simular o valor de uma variável aleatória com f.m.p. $p_j, j \geq 0$ ou $q_j, j \geq 0$, e que desejamos simular a variável aleatória X com f.m.p.:

$$\Pr(X = j) = \alpha p_j + (1 - \alpha) q_j, \quad j \geq 0, 0 < \alpha < 1.$$

Se $X_1 \sim p_j$ e $X_2 \sim q_j$, então podemos definir

$$X = \begin{cases} X_1, & \text{com probabilidade } \alpha, \\ X_2, & \text{com probabilidade } 1 - \alpha, \end{cases}.$$

Assim, X terá exatamente a função de probabilidade acima.

Exemplo

Suponha que desejamos gerar o valor de uma variável aleatória X tal que

$$p_j = \Pr(X = j) = \begin{cases} 0.05, & \text{para } j = 1, 2, 3, 4, 5, \\ 0.15, & \text{para } j = 6, 7, 8, 9, 10, \end{cases}$$

Note que $p_j = 0.5 \times p_j^{(1)} + 0.5 \times p_j^{(2)}$, em que

$$p_j^{(1)} = 0.1, \quad j = 1, \dots, 10 \quad \text{e} \quad p_j^{(2)} = \begin{cases} 0, & \text{para } j = 1, 2, 3, 4, 5, \\ 0.2, & \text{para } j = 6, 7, 8, 9, 10, \end{cases}$$

podemos realizar essa simulação gerando primeiro um número aleatório $U \sim U(0, 1)$ e então:

- Se $U < 0.5$, gerar X de uma uniforme discreta sobre $\{1, 2, \dots, 10\}$.
- Caso contrário ($U \geq 0.5$), gerar X de uma uniforme discreta sobre $\{6, 7, 8, 9, 10\}$.

Algoritmo

Passo 1. Gerar $U_1 \sim U(0, 1)$.

Passo 2. Gerar $U_2 \sim U(0, 1)$.

Passo 3. Se $U_1 < 0.5$, definir $X = \text{Int}(10U_1) + 1$. Caso contrário, definir $X = \text{Int}(5U_2) + 6$.

Se $F_i, i = 1, \dots, n$ são funções de distribuição e $\alpha_i, i = 1, \dots, n$ são números não negativos cuja soma é 1, então a função de distribuição:

$$F(x) = \sum_{i=1}^n \alpha_i F_i(x),$$

é uma **mistura**, ou uma **composição**, das funções de distribuição F_i , $i = 1, \dots, n$.

Uma maneira de simular a partir de F é primeiro simular uma variável aleatória I , igual a i com probabilidade α_i , $i = 1, \dots, n$, e então simular a partir da distribuição F_I (Isto é, se o valor simulado de I for $I = j$, então a segunda simulação é feita a partir de F_j). Essa abordagem para simular de F é frequentemente chamada de **método de composição**.

3.2 Caso Contínuo

A geração de variáveis aleatórias contínuas baseia-se quase inteiramente na existência de um fluxo de números independentes e identicamente distribuídos (i.i.d.) vindos de uma distribuição Uniforme $(0,1)$.

3.2.1 Método da Transformação Inversa

O método da inversão é a técnica mais direta para gerar variáveis aleatórias. Ele se baseia na Transformada Integral de Probabilidade: se X é uma variável aleatória contínua com função de distribuição acumulada (FDA), $F(x)$, então a variável $U = F(X)$ segue uma distribuição Uniforme $(0,1)$.

Algoritmo

1. Gerar $U \sim U(0,1)$;
2. Definir $X = F^{-1}(U)$, onde $F^{-1}(\cdot)$ é a função quantílica (inversa da FDA).

3.2.2 Método da Aceitação e Rejeição

Quando a inversa da CDF não possui uma forma fechada (como na distribuição Beta ou Normal), utilizamos métodos indiretos. O método da rejeição requer uma densidade instrumental $g(x)$ que seja fácil de simular e uma constante c tal que a razão entre a densidade alvo $f(x)$ e $g(x)$ seja limitada: $f(x)/g(x) \leq c$.

Algoritmo

1. Simula $Y \sim g$ e $U \sim U(0,1)$ de forma independente;

2. Aceita $X = Y$ se $U \leq \frac{f(Y)}{cg(Y)}$.
3. Caso contrário, rejeita Y e retornar ao passo 1.

A eficiência deste método é dada por $1/c$, que representa a probabilidade de aceitação em cada tentativa.

3.2.3 Método da Composição (Misturas)

Algumas distribuições complexas podem ser decompostas como uma mistura de distribuições mais simples. Se a densidade alvo puder ser escrita como $f(x) = \sum p_i f_i(x)$, simulamos primeiro um índice I com probabilidade p_i e depois geramos X a partir da densidade f_I selecionada.

3.2.3.1 Exemplo:

A distribuição de t de Student pode ser representada como uma mistura de Normais onde a variância é controlada por uma variável Qui-quadrado.

4 Métodos de Monte Carlo

Os métodos de Monte Carlo formam uma classe de algoritmos baseados em simulação aleatória. Eles surgiram no final da década de 1940, especialmente com os trabalhos de John von Neumann, Stanislaw Ulam, Enrico Fermi e Nicholas Metropolis. O nome deriva da cidade famosa por seus cassinos.

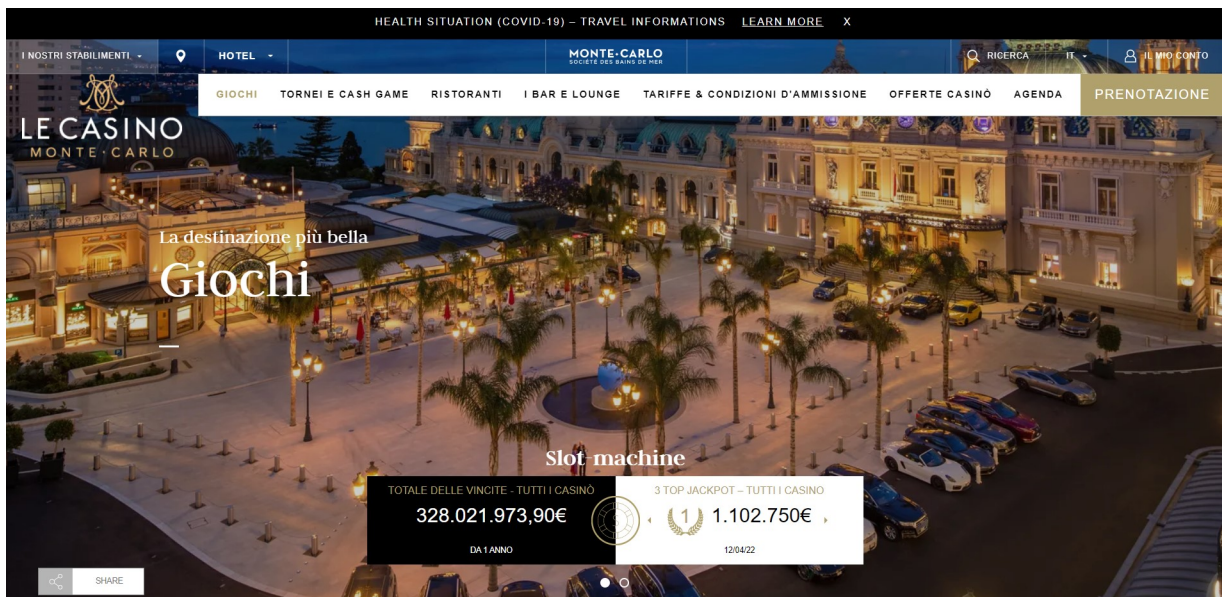


Figura 4.1: Cassino — Aleatoriedade como metáfora

Na Estatística, os métodos de Monte Carlo são utilizados para:

- estimar parâmetros da distribuição amostral;
- calcular EQM (erro quadrático médio);
- avaliar cobertura de intervalos de confiança;
- estimar probabilidades de erro tipo I;
- estimar o nível descritivo de um teste de hipóteses;
- estimar poder de testes de hipóteses.

4.1 Uso de Números Aleatórios na Avaliação de Integrais

Uma das primeiras aplicações do uso de números aleatórios foi na resolução de integrais. Considere uma função $g(x)$ e suponha que desejamos calcular uma integral de interesse.

$$\theta = \int_0^1 g(x)dx.$$

Para calcular o valor da integral, observe que, se U é uma variável aleatória com distribuição uniforme no intervalo $(0, 1)$, então podemos reescrever a integral da seguinte forma:

$$\theta = E[g(U)].$$

Se $U_1, U_2, \dots, U_n$ são variáveis aleatórias independentes e uniformes em $(0, 1)$, então as variáveis $g(U_1), g(U_2), \dots, g(U_n)$ são independentes e identicamente distribuídas, todas com esperança igual θ (o valor da integral). Assim, pelo **Teorema da Lei Forte dos Grandes Números**, temos que, com probabilidade 1,

$$\frac{1}{n} \sum_{i=1}^n g(U_i) \rightarrow \theta \quad \text{quando } n \rightarrow \infty.$$

Assim, podemos aproximar o valor da integral gerando um grande número de pontos aleatórios u_i no intervalo $(0, 1)$ e tomando como estimativa a média dos valores $g(u_i)$. Esse procedimento de aproximação de integrais é conhecido como método de **Monte Carlo**.

Se quisermos calcular uma integral mais geral, podemos aplicar a mesma ideia: transformar o problema em uma esperança matemática e, em seguida, aproximá-la por meio de médias amostrais obtidas a partir de números aleatórios. Considere:

$$\theta = \int_a^b g(x)dx.$$

Se quisermos calcular a integral em um intervalo genérico (a, b) , fazemos a substituição

$$u = \frac{x-a}{b-a}, \quad du = \frac{dx}{b-a},$$

o que nos permite reescrevê-la como

$$\theta = \int_a^b g(x)dx = (b-a) \int_0^1 g(a + (b-a)u)du.$$

Definindo

$$h(u) = (b-a)g(a + (b-a)u),$$

temos

$$\theta = \int_0^1 h(u)du.$$

Assim, podemos aproximar θ gerando números aleatórios $u_1, u_2, \dots, u_n \sim U(0, 1)$ e tomando como estimativa a média

$$\frac{1}{n} \sum_{i=1}^n h(u_i).$$

Agora, se nosso objetivo é calcular a integral:

$$\theta = \int_0^\infty g(x)dx.$$

Fazendo a mudança de variável

$$u = \frac{1}{x+1}, \quad du = \frac{-dx}{(x+1)^2} = -u^2 dx.$$

Logo,

$$dx = -\frac{du}{u^2},$$

e a integral resultante é

$$\theta = \int_0^1 h(u)du,$$

com

$$h(u) = \frac{g\left(\frac{1}{u} - 1\right)}{u^2}.$$

A utilidade de empregar números aleatórios para aproximar integrais torna-se ainda mais evidente no caso de integrais multidimensionais. Suponha que g seja uma função com argumento n -dimensional e que estejamos interessados em calcular:

$$\theta = \int_0^1 \int_0^1 \dots \int_0^1 g(x_1, x_2, \dots, x_n) dx_1 dx_2 \dots dx_n.$$

Observe que θ pode ser expresso como o seguinte valor esperado:

$$\theta = E[g(U_1, U_2, \dots, U_n)],$$

em que $U_1, U_2, \dots, U_n$ são variáveis aleatórias independentes uniformemente distribuídas no intervalo $(0, 1)$. Assim, se gerarmos k conjuntos independentes, cada um formado por n variáveis aleatórias independentes com distribuição uniforme em $(0, 1)$, então as variáveis

$$g(U_{i1}, U_{i2}, \dots, U_{in}), \quad i = 1, 2, \dots, k,$$

serão independentes e identicamente distribuídas, todas com esperança igual a θ (o valor da integral). Portanto, podemos estimar θ por meio da média amostral:

$$\hat{\theta} = \frac{1}{k} \sum_{i=1}^k g(U_{i1}, U_{i2}, \dots, U_{in}).$$

Exemplo

Nosso objetivo é encontrar o valor aproximado da integral: $\int_0^1 x^3 dx = 0.25$.

```
set.seed(2025) #fixando a semente
n <- 100000 #quantidade de valores uniformes gerados
u <- runif(n) #numeros uniformes (0,1)
mean(u^3)
```

```
[1] 0.2503221
```

```
import numpy as np
np.random.seed(2025)
n = 100000
u = np.random.uniform(0, 1, n)
resultado = np.mean(u**3)

print(resultado)
```

0.2508972398766188

Exercícios

1. Se $x_0 = 5$ e $x_n = 3x_{n-1} \bmod 150$, encontre $x_1, x_2, \dots, x_{30}$.
2. Se $x_0 = 3$ e $x_n = (5x_{n-1} + 7) \bmod 200$, encontre $x_1, x_2, \dots, x_{10}$.
3. Compare sua estimativa com o valor exato:
 - $\int_0^1 \exp\{e^x\} dx$.
 - $\int_0^1 (1 - x^2)^{3/2} dx$.
 - $\int_{-2}^2 \exp\{x + x^2\} dx$.
 - $\int_0^\infty x(1 + x^2)^{-2} dx$.
 - $\int_{-\infty}^\infty \exp\{-x^2\} dx$.
 - $\int_0^1 \int_0^1 \exp\{(x + y)^2\} dy dx$.
 - $\int_0^\infty \int_0^x \exp\{-(x + y)\} dy dx$.
4. Use simulação para encontrar o valor aproximado de $Cov(U, e^U)$, em que U é uniforme em $(0, 1)$. Compare seu resultado com o valor exato.

Estimativa de Probabilidades via Monte Carlo

O método de Monte Carlo permite aproximar integrais complexas e, consequentemente, probabilidades, ao tratar o problema como o cálculo de uma esperança matemática.

Para qualquer evento A , a probabilidade $P(X \in A)$ pode ser expressa como a esperança da função indicadora $I_A(X)$, em que $I_A(X) = 1$ se $x \in A$ e 0 caso contrário. Pela Lei Forte dos Grandes Números, a média aritmética de uma amostra independente e identicamente distribuída (i.i.d.) converge para o valor real da esperança conforme o tamanho da amostra, n , aumenta

$$\hat{P}_n = \frac{1}{n} \sum_{i=1}^n I_A(X_i) \rightarrow P(X \in A).$$

Precisão e Erro Padrão

Como $\hat{P}$ é a média de variáveis Bernoulli, sua variância teórica é dada por $Var(\hat{P}) = \frac{P(1-P)}{n}$. De acordo com o Teorema Central do Limite, para um n suficientemente grande, o erro da estimativa distribui-se aproximadamente como uma distribuição Normal, permitindo calcular o erro padrão da simulação.

4.1.0.0.1 * Exemplo: $P(X > 2.5)$, com $X \sim N(0,1)$

No R, a estimativa é realizada de forma eficiente através da vetorização, evitando laços lentos. Para eventos nas caudas (eventos raros), é essencial monitorar a estabilidade da estimativa.

```
set.seed(20262) # Garante a reprodutibilidade
n <- 1e6
x <- rnorm(n)

# Estimativa de Monte Carlo
indicadora <- x > 2.5
p_hat <- mean(indicadora)

# Cálculo do Erro Padrão
erro_padrao <- sqrt(p_hat * (1 - p_hat) / n)

# Resultados
cat("Estimativa:", p_hat, "\n")
```

Estimativa: 0.006225

```
cat("Erro Padrão:", erro_padrao, "\n")
```

Erro Padrão: 7.865271e-05

```
cat("Intervalo de Confiança (95%): [", p_hat - 1.96*erro_padrao, ",", p_hat + 1.96*erro_padrao, "]\n")
```

Intervalo de Confiança (95%): [0.006070841 , 0.006379159]

Embora o método de Monte Carlo direto seja robusto, ele pode ser ineficiente para probabilidades muito pequenas, exigindo um n astronômico para obter precisão relativa. Nesses casos, técnicas como a Amostragem de Importância podem ser utilizadas para reduzir a variância, simulando de uma distribuição que enfatize a região de interesse (neste caso, valores $> 2,5$)

4.1.1 Erro e Convergência de Monte Carlo

O objetivo fundamental da integração de Monte Carlo é estimar a esperança matemática $I = E_f[h(X)] = \int_{\mathcal{X}} h(x)f(x)dx$. O estimador de Monte Carlo, $\bar{h}_n$, é a média aritmética de uma amostra $X_1, \dots, X_n$ gerada a partir da densidade f :

$$\bar{h}_n = \frac{1}{n} \sum_{i=1}^n h(X_i).$$

Note que a validade desta aproximação possui pilares da teoria das probabilidades.

1. Lei Forte dos Grandes Números: garante que $\bar{h}_n$ converge quase certamente para o valor real I conforme $n \rightarrow \infty$.
2. Teorema Central do Limite (TCL): Afirma que, para n grande, o erro de integração $\bar{h}_n - I$ possui distribuição aproximadamente normal com média 0 e variância σ^2/n .

Como a variância populacional $\sigma^2 = \text{Var}(h(X))$ é geralmente desconhecida, nós estimamos a partir da amostra para avaliar a precisão do método:

com estimador de Monte Carlo: $\bar{h}_n$. A Variância do estimador de Monte Carlo é dada por:

$$\widehat{\text{Var}}(h(X)) = \frac{1}{n} \sum_{j=1}^n (h(x_j) - \bar{h}_n)^2.$$

Consequentemente, a variância do estimador da média, $\text{Var}(\bar{h}_n)$ é estimada por:

$$v_n = \frac{\widehat{\text{Var}}(\bar{h}_n)}{n} = \frac{1}{n^2} \sum_{j=1}^n (h(x_j) - \bar{h}_n)^2.$$

O erro padrão, $\sqrt{v_n}$, é a medida padrão da variabilidade do estimador. É importante notar que a precisão de Monte Carlo melhora a uma taxa de $O(n^{-1/2})$, o que significa que para reduzir o erro pela metade, é necessário quadruplicar o número de simulações.

Uma prática recomendada é monitorar a evolução da estimativa e das bandas de erro conforme n aumenta. As bandas baseadas em $\pm 1.96\sqrt{v_n}$ representam um intervalo de confiança de 95% para a estimativa em cada ponto da iteração.

Nota de cautela: Estas bandas indicam a confiança na estimativa se a simulação fosse interrompida naquele valor de n . Elas não garantem que a trajetória inteira do estimador permanecerá dentro desses limites em uma nova execução.

```
set.seed(1234)
n <- 1e3
x <- runif(n)
h <- (cos(50*x) + sin(20*x))^2

# Estimativas acumuladas (vetorizadas para eficiência no R)
hbn <- cumsum(h) / (1:n)
# Variância acumulada do estimador da média
# Nota: usando a fórmula de soma de quadrados para v_n
vn <- (cumsum(h^2) / (1:n) - hbn^2) / (1:n)

# Intervalo de Confiança Assintótico (95%)
erro_padrao <- sqrt(vn)
LI <- hbn - 1.96 * erro_padrao
LS <- hbn + 1.96 * erro_padrao

# Valor verdadeiro da integral para comparação
I_true <- 0.965201

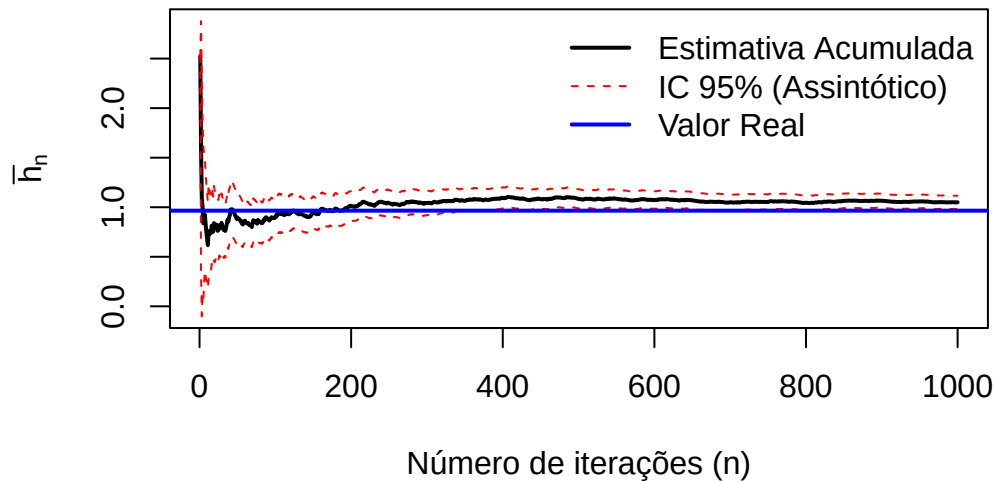
# Visualização da Convergência
plot(hbn, type = "l", lwd = 2, xlab = "Número de iterações (n)",
     ylab = expression(bar(h)[n]),
     main = "Convergência de Monte Carlo com Bandas",
     ylim = range(c(LI, LS), na.rm = TRUE))

lines(LI, col = "red", lty = 2)
lines(LS, col = "red", lty = 2)
```

```
abline(h = I_true, col = "blue", lwd = 2)

legend("topright",
      legend = c("Estimativa Acumulada", "IC 95% (Assintótico)", "Valor Real"),
      col = c("black", "red", "blue"), lty = c(1, 2, 1), lwd = c(2, 1, 2), bty = "n")
```

Convergência de Monte Carlo com Bandas



4.2 Simulações de Monte Carlo em Inferência

Simulações de Monte Carlo têm como objetivo estudar as propriedades de técnicas de inferência estatística. No essencial, uma simulação de Monte Carlo funciona aplicando essas técnicas a amostras repetidamente geradas a partir de um processo de geração de dados previamente especificado.

O pacote `tidyMC` busca contemplar e simplificar todo o fluxo de trabalho necessário para executar uma simulação de Monte Carlo, seja em um contexto acadêmico ou profissional. Assim, o `tidyMC` fornece funções para as seguintes tarefas:

- Executar uma simulação de Monte Carlo para uma função definida pelo usuário sobre uma grade de parâmetros, utilizando `future_mc()`.
- Resumir os resultados por meio de funções de resumo (opcionais) definidas pelo usuário, usando `summary.mc()`.
- Criar gráficos dos resultados da simulação de Monte Carlo, que podem ser modificados pelo usuário, por meio de `plot.mc()` e `plot.summary.mc()`.

- Gerar uma tabela em LaTeX resumindo os resultados da simulação de Monte Carlo usando `tidy_mc_latex()`.

4.2.1 Instalação

O pacote `tidyMC` está disponível no CRAN. Para baixar a versão do CRAN basta:

```
install.packages("tidyMC")
```

Em seguida, o pacote pode ser carregado com:

```
library(tidyMC)
```

Além disso, os seguintes pacotes serão utilizados:

```
library(magrittr)
library(ggplot2)
library(kableExtra)
```

4.3 Execute sua primeira Simulação de Monte Carlo com `future_mc()`

A função `future_mc()` permite executar uma simulação de Monte Carlo para uma função definida pelo usuário e para um conjunto especificado de parâmetros.

O primeiro argumento de `future_mc()` é `fun`, que deve ser uma função capaz de realizar três tarefas para uma única repetição e uma única combinação de parâmetros:

- gerar os dados,
- aplicar o método de interesse,
- avaliar o resultado obtido.

A função `future_mc()` se encarrega de gerar automaticamente os laços sobre as grades de parâmetros desejadas, bem como repetir o experimento de Monte Carlo para cada combinação de parâmetros.

Considere o seguinte exemplo de função `fun`, que realiza exatamente as tarefas necessárias: geração dos dados, aplicação do método e avaliação dos resultados.


```
test_func <- function(param1 = 10, param2 = 2, n = 100) {
  data <- rnorm(n, mean = param1, sd = param2)
  stat <- mean(data)
  stat_2 <- sum((data - mean(data))^2)/length(data)
  dev_mu <- stat - param1
  dev_sigma2 <- stat_2 - param2^2
  dev_eqm_mu <- (stat - param1)^2
  dev_eqm_sigma2 <- (stat_2 - param2^2)^2
  return(list(mean = stat, var = stat_2, bmu = dev_mu, bsig2 = dev_sigma2,
             eqmmu = dev_eqm_mu,
             eqmsigma2 = dev_eqm_sigma2))
}
```

O segundo argumento de `future_mc()` é `repetitions`, que deve ser um número inteiro especificando o total de iterações da simulação de Monte Carlo.

O terceiro argumento, `param_list`, deve ser uma lista cujos componentes têm nomes correspondentes aos parâmetros da função `fun` que deverão variar ao longo da simulação. Cada componente dessa lista é um vetor contendo os valores desejados da grade para aquele parâmetro.

Considere o exemplo a seguir para `param_list`. Observe que seus componentes recebem nomes que correspondem exatamente aos parâmetros da função `test_func()`: `param` e `x1`, respectivamente:

```
param_list <- list(
  n = c(10, 50, 90)
)
```

A função `future_mc()` cria automaticamente todas as combinações possíveis de parâmetros especificadas em `param_list` e executa a simulação de Monte Carlo para cada uma dessas combinações. Se você preferir não rodar a simulação para todas as combinações possíveis, pode fornecer `param_table`, que deve ser um *data.frame* ou *data.table* contendo apenas as combinações de parâmetros de interesse.

O argumento `...` permite especificar outros argumentos da função `fun` que não estão em `param_list`. Esses argumentos permanecerão fixos ao longo de todas as combinações de parâmetros.

Requisitos formais que `fun` deve satisfazer:

1. Os argumentos presentes em `param_list` devem ser escalares;
2. Toda variável usada dentro de `fun` deve ser definida internamente ou fornecida via `...`;

3. O valor retornado por `fun` deve ser uma lista nomeada;
4. Os nomes dos valores retornados e os nomes dos argumentos presentes em `param_list` não podem coincidir.

Se a função `fun` retorna uma lista nomeada de escalares, o usuário pode utilizar `summary()` para resumir todos os resultados da simulação de Monte Carlo.

```
set.seed(1234)
test_mc <- future_mc(
  fun = test_func,
  repetitions = 1000,
  param_list = param_list,
  param1 = 10,
  param2 = 2
)
```

Running single test-iteration for each parameter combination...

Test-run successfull: No errors occurred!

Running whole simulation: Overall 3 parameter combinations are simulated ...

Simulation was successfull!
Running time: 00:00:00.746093

```
summary(test_mc)
```

Results for the output mean:

n=10: 9.991484
n=50: 9.992515
n=90: 9.997018

Results for the output var:

n=10: 3.678279
n=50: 3.916758
n=90: 3.949042

Results for the output bmu:

```
n=10: -0.008515738
n=50: -0.007484689
n=90: -0.002981764
```

Results for the output bsig2:

```
n=10: -0.3217209
n=50: -0.08324241
n=90: -0.0509576
```

Results for the output eqmmu:

```
n=10: 0.3646171
n=50: 0.08530073
n=90: 0.04476774
```

Results for the output eqmsigma2:

```
n=10: 3.350142
n=50: 0.5993374
n=90: 0.3374168
```

A função `plot()` retorna uma lista de objetos das classes `gg` e `ggplot2`. Essa lista conterá um gráfico de linhas para cada saída produzida pela função `fun`, representando as trajetórias dos resultados para os diferentes cenários da simulação.

```
plot(summary(test_mc))
```

Para gerar uma tabela em LaTeX basta:

```
tidy_mc_latex(
  x = summary(test_mc)
) %>%
print()
```

5 Bootstrap e Jackknife

5.1 Bootstrap

Ao longo deste material, utilizamos simulações de Monte Carlo para demonstrar propriedades estatísticas de estimadores. Simulamos processos de geração de dados no computador e, em seguida, examinamos diretamente os resultados.

Este capítulo explica como técnicas de simulação intensivas em computação podem ser aplicadas a **uma única amostra** para estimar a distribuição amostral de uma estatística. Esses procedimentos, cada vez mais populares, são conhecidos como **métodos bootstrap**. Eles podem ser usados para corroborar resultados baseados na teoria padrão ou fornecer respostas quando métodos convencionais falham.

A expressão inglesa “*pull yourself up by your bootstraps*” descreve a situação de alguém que consegue superar dificuldades **sozinho**, apesar de recursos limitados. A frase vem do livro *The Surprising Adventures of Baron Munchausen*, de Rudolph Erich Raspe. O barão conta uma série de histórias fantásticas sobre suas viagens, incluindo feitos impossíveis e fugas ousadas. Bradley Efron escolheu o termo “bootstrap” para descrever um esquema de reamostragem que estava desenvolvendo porque “o uso do termo bootstrap deriva da expressão *to pull oneself up by one’s own bootstrap*... (O Barão havia caído no fundo de um lago profundo. Quando tudo parecia perdido, ele teve a ideia de se erguer puxando as próprias alças das botas.)” (Efron e Tibshirani (1993), p. 5).

Em estatística e em outras áreas, **bootstrapping** passou a significar **reamostrar repetida e aleatoriamente a partir de uma amostra original**, usando cada amostra reamostrada para calcular uma estatística. A distribuição empírica resultante dessa estatística é então examinada e interpretada como uma aproximação da verdadeira distribuição amostral.

Nota

O vínculo entre bootstrap e Monte Carlo é evidente: ambos se baseiam em amostragem repetitiva e exame direto dos resultados. A grande diferença, porém, é que o *bootstrap* usa a **amostra original** como população da qual se reamostra, enquanto as simulações de Monte Carlo partem de um **processo de geração de dados especificado** (com valores conhecidos dos parâmetros). Enquanto Monte Carlo serve para testar estimadores, os métodos *bootstrap* podem ser usados para estimar a variabilidade de uma estatística e o

formato de sua distribuição amostral.

O Bootstrap pode ser utilizado com o propósito de estimar erros padrão, viés de estimadores, construir intervalos de confiança, testes de hipóteses, entre outros. Pode ser utilizando sob duas abordagens: **paramétrica** e **não paramétrica**. A abordagem paramétrica exige um modelo estatístico, enquanto que na abordagem não paramétrica não há suposição de modelo estatístico; toma-se por base uma distribuição empírica que atribui probabilidade $1/n$ para cada um dos n elementos da amostra. Algumas referências importantes neste tema são: Efron (1979), Wu (1986), Fisher & Hall (1989), Fredman (1986), Efron & Tibshirani (1993), Horowitz (1997), Davison & Hinkley (1997).

5.1.1 Acurácia da média amostral

Experimento com 16 ratos, divididos em dois grupos: um grupo recebeu o tratamento e um outro grupo não recebeu o tratamento (controle). O tempo de sobrevivência (dias) é apresentado para cada um dos ratos. O tratamento prolonga a vida?

grupo	tempo 1	tempo 2	tempo 3	n	média	$\widehat{ep}(\text{média})$
tratamento (X)	94	197	16	7	86,86	25,24
	38	99	141			
	23					
controle (Y)	52	104	146	9	56,22	14,14
	10	51	30			
	40	27	46			
diferença					30,63	28,93

Alguns pontos importantes são:

- A resposta à pergunta dependerá de quão acurado(s) é(são) o(s) estimador(es).
- O erro padrão é uma medida (muito usual) de acurácia de estimador.
- erro padrão estimado para a média amostral $\bar{X}$

$$\widehat{ep}(\bar{X}) = \sqrt{\frac{s^2}{n}},$$

em que $s^2 = \sum_{i=1}^n (x_i - \bar{x})^2 / (n - 1)$.

- erro padrão de qualquer estimador é definido pela raiz quadrada de sua variância

Além disso, temos que:

- erro padrão pequeno $\rightarrow$ acurácia alta
- erro padrão grande $\rightarrow$ acurácia baixa
- acurácia alta (baixa) indica que o estimador apresenta valores próximos (distantes) ao seu valor esperado
- espera-se que 68% dos valores do estimador estejam a menos de um erro padrão do seu valor esperado, e 95%, a menos de dois erros padrões
- erro padrão da diferença $(\bar{X} - \bar{Y})$:

$$28.93 = \sqrt{25.24^2 + 14.14^2}.$$

Resposta à pergunta: a diferença observada 30.63 é somente $30.63/28.93=1.05$ erros padrões (estimados) maior que zero, indicando um resultado não significativo, ou seja, o tratamento não aumenta o tempo médio de vida (considerando a teoria dos testes de hipóteses).

O erro padrão para o estimador média amostral apresenta fórmula conhecida, mas há casos em que não dispomos de fórmulas. Suponha que haja interesse em comparar os dois grupos de ratos em relação aos tempos medianos. Temos: $md(X)=94$ e $md(Y)=46$. A diferença é 48, maior que a diferença para as médias. Com base na mediana, o tratamento prolonga a vida?

5.1.2 Estimativa bootstrap do erro padrão

Considerações:

- $\tilde{x} = (x_1, x_2, \dots, x_n)$: vetor de dados observado (amostra original de tamanho n)
- $s(\tilde{x})$: estatística de interesse (por exemplo, média amostral)
- Uma amostra bootstrap $\tilde{x}^* = (x_1^*, x_2^*, \dots, x_n^*)$ é obtida pela amostragem aleatória de tamanho n , com reposição, de $\tilde{x}$. Por exemplo, com $n = 7$, poderíamos obter $\tilde{x}^* = (x_1^*, x_2^*, \dots, x_n^*) = (x_5, x_7, x_5, x_4, x_7, x_3, x_1)$.

O algoritmo bootstrap

- gerar B amostras bootstrap independentes: $\tilde{x}^{*1}, \tilde{x}^{*2}, \dots, \tilde{x}^{*B}$, cada uma de tamanho n e $50 \leq B \leq 200$.
- calcular $s(\tilde{x}^{*b})$, $b = 1, 2, \dots, B$. $s(\tilde{x}^{*b})$ é denominada réplica bootstrap de $s(\tilde{x})$
- calcular $\hat{e}p_{boot} = \hat{e}p_B = \sqrt{\frac{\sum_{b=1}^B [s(\tilde{x}^{*b}) - s(\tilde{x})]^2}{B-1}}$, em que $s(\tilde{x}) = \frac{\sum_{b=1}^B s(\tilde{x}^{*b})}{B}$

Sintaxe do R para calcular a estimativa bootstrap do erro padrão da média do tempo de sobrevivência dos ratos do grupo tratamento.

```
set.seed(1234)
# grupo tratamento (amostra original)
x <- c(94, 197, 16, 38, 99, 141, 23)
n <- length(x)
s <- 0 # estatística de interesse
B <- 50 # no. de amostras bootstrap

for(i in 1:B){
  # réplica bootstrap para o estimador média
  s[i] <- mean(sample(x,n,replace=TRUE))
}
# estimativa bootstrap para o erro padrão da média
ep <- sd(s); ep
```

[1] 27.41873

Estimativas bootstrap do erro padrão da média e da mediana do tempo de sobrevivência dos ratos do grupo tratamento. A mediana é menos acurada (erros padrões maiores) que a média para esse conjunto de dados.

B	50	100	250	500	1000	∞
média	19.72	23.63	22.32	23.79	23.02	23.36
mediana	32.21	36.35	34.46	36.72	36.48	37.83

Formalização:

- $\tilde{X} = X_1, X_2, \dots, X_n$: amostra aleatória de uma f.d.a. F
- $\tilde{x} = (x_1, x_2, \dots, x_n)$: amostra aleatória observada de F
- $\Theta = t(F)$: parâmetro (Θ uma função de F)
- $\hat{\Theta} = s(\tilde{X})$: estimador para Θ
- $\hat{\theta} = s(\tilde{x})$: estimativa para Θ
- Quão acurado é o estimador $\hat{\Theta}$?

- Seja $\hat{F}$ a distribuição empírica que atribui a probabilidade $1/n$ para cada valor observado x_i , $i = 1, 2, \dots, n$. A amostra bootstrap $\tilde{x}^*$ é definida como a amostra aleatória com reposição de tamanho n extraída de $\hat{F}$.

$$\tilde{x}^* = (x_1^*, x_2^* \dots, x_n^*)$$

$$\hat{F} \longrightarrow (x_1^*, x_2^* \dots, x_n^*)$$

Nota:

- $\tilde{x}^*$: o símbolo $*$ indica que a amostra não é a original $\tilde{x}$, mas uma versão aleatorizada(reamostrada) de $\tilde{x}$
- a cada amostra bootstrap corresponde uma réplica bootstrap de $\hat{\theta}$, $\hat{\theta}^* = s(\tilde{x}^*)$
- $ep_F(\hat{\Theta})$ é estimado por $ep_{\hat{F}}(\hat{\Theta}^*)$, chamado **estimador bootstrap ideal** para o erro padrão $\hat{\Theta}$
- Raramente faz-se necessário $B \geq 200$ para estimar erro padrão; valores (muito) maiores são necessários, por exemplo, para IC bootstrap.
- $\lim_{B \rightarrow \infty} \hat{ep}_B = ep_{\hat{F}} = ep_{\hat{F}}(\hat{\Theta}^*)$
- O estimador bootstrap ideal $ep_{\hat{F}}(\hat{\Theta}^*)$ e sua aproximação $\hat{ep}_B$ são chamados **estimadores bootstrap não-paramétricos**, pois baseiam-se em $\hat{F}$, o estimador não-paramétrico de F .
- total de amostras bootstrap distintas (combinação com repetição):

$$\binom{2n-1}{n}.$$

- No R: ver as funções `factorial()`, `choose()`, `combn()`, `combinations()`.

Exemplo

Dados de faculdades americanas de direito. População: $N = 82$ faculdades. Amostra aleatória: $n = 15$ faculdades. Variáveis analisadas: LSAT (escore médio em um teste), GPA (pontuação média na faculdade).

escola	LSAT	GPA	escola	LSAT	GPA
1	576	3,39	9	651	3,36
2	635	3,30	10	605	3,13

escola	LSAT	GPA	escola	LSAT	GPA
3	558	2,81	11	653	3,12
4	578	3,03	12	575	2,74
5	666	3,44	13	545	2,76
6	580	3,07	14	572	2,88
7	555	3,00	15	594	2,96
8	661	3,43			

- Façamos $Y=LSAT$ e $Z=GPA$. A estatística (estimador) de interesse é o coeficiente de correlação amostral entre as variáveis Y e Z :

$$\hat{\Theta} = corr(Y, Z) = \frac{Cov(Y, Z)}{DP(Y).DP(Z)} = \frac{\sum_{i=1}^n (Y_i - \bar{Y})(Z_i - \bar{Z})/n}{DP(Y).DP(Z)}$$

- Para os dados observados, a estimativa do coeficiente de correlação amostral é 0.776. Quão acurado é o estimador?

Tabela 5.4: **Estimativas bootstrap do erro padrão para $\hat{\Theta} = corr(Y, Z)$**

B	25	50	100	200	400	800	1600	3200
$\hat{e}p_B$	0,140	0,142	0,151	0,143	0,141	0,137	0,133	0,132

i Nota

No caso de valores extremos inflacionarem fortemente $\hat{e}p_B$, uma medida mais robusta para o estimador bootstrap do erro padrão é desejável. (ver Efron & Tibshirani (1993)).

i Nota

Inferências baseadas na distribuição normal são “questionáveis” quando o histograma das réplicas bootstrap indica forte assimetria.

Exercícios:

- (1) Considere $B = 3200$. Para cada uma das 3200 amostras bootstrap, obtenha a réplica bootstrap $\hat{\theta}^* = corr(y^*, z^*)$. Faça o histograma das réplicas.
- (2) A Tabela 3.2, p. 21 do livro texto, apresenta os dados populacionais das 82 faculdades. Selecione 3200 amostras aleatórias de tamanho $n = 15$. Para cada uma dessas amostra calcule o coeficiente de correlação e faça o histograma.

5.1.3 Bootstrap Paramétrico

O estimador bootstrap paramétrico do erro padrão é definido por

$$ep_{\hat{F}_{par}}(\hat{\Theta}^*),$$

em que $\hat{F}_{par}$ é um estimador de F derivado do modelo paramétrico para os dados.

Para os dados das faculdades: Vamos supor que a população (LSAT, GPA) possa ser descrita por um modelo paramétrico normal bivariado F . Estimamos F por $\hat{F}_{normal}$, que denota a f.d.a. de uma normal bivariada com vetor de médias e matriz de covariâncias $(\bar{y}, \bar{z})$ e $\frac{1}{14} \begin{pmatrix} \sum (y_i - \bar{y})^2 & \sum (y_i - \bar{y})(z_i - \bar{z}) \\ \sum (y_i - \bar{y})(z_i - \bar{z}) & \sum (z_i - \bar{z})^2 \end{pmatrix}$.

O estimador bootstrap paramétrico do erro padrão da correlação $\hat{\Theta}$ será dado por $ep_{\hat{F}_{normal}}(\hat{\Theta}^*)$. Esse estimador bootstrap ideal será aproximado por $\hat{ep}_B$ (conforme algoritmo a seguir).

O algoritmo bootstrap:

- extrair B amostras de tamanho n de $\hat{F}_{par}$: $\tilde{x}^{*1}, \tilde{x}^{*2}, \dots, \tilde{x}^{*B}$
- calcular $s(\tilde{x}^{*b})$, $b = 1, 2, \dots, B$. $s(\tilde{x}^{*b})$ é a réplica bootstrap de $s(x)$
- calcular $\hat{ep}_{boot} = \hat{ep}_B = \sqrt{\frac{\sum_{b=1}^B [s(\tilde{x}^{*b}) - s(\cdot)]^2}{B-1}}$, em que $s(\cdot) = \frac{\sum_{b=1}^B s(\tilde{x}^{*b})}{B}$

No exemplo das faculdades, assumindo o modelo normal bivariado, extraímos B amostras de tamanho $n = 15$ de $\hat{F}_{normal}$, calculamos o coeficiente de correlação para cada amostra e, por fim, calculamos o desvio padrão desses coeficientes de correlação. Usando $B = 3200$ encontramos $\hat{ep}_B = 0.124$, que é próximo ao valor 0.131 obtido com o bootstrap não-paramétrico.

A fórmula teórica para o erro padrão do coeficiente de correlação é $\frac{1 - \hat{\Theta}^2}{\sqrt{n-3}}$, com $\hat{\Theta} = \text{corr}(Y, Z)$. Vimos que $\hat{\theta} = 0.776$, o que resulta a estimativa 0.115 para o erro padrão do coeficiente de correlação entre as variáveis $Y=\text{LSAT}$ e $Z=\text{GPA}$.

Transformação de Fisher para o coeficiente de correlação $\hat{\Theta}$: $\hat{\zeta} = 0.5 \log \left(\frac{1 + \hat{\Theta}}{1 - \hat{\Theta}} \right)$. Assim, $\hat{\zeta}$ tem distribuição aproximadamente normal com média $0.5 \log \left(\frac{1 + \Theta}{1 - \Theta} \right)$ e variância $\frac{1}{n-3}$. O erro padrão para $\hat{\zeta}$ é $\sqrt{\frac{1}{n-3}}$. Para o exemplo das faculdades, o valor é $\frac{1}{\sqrt{12}} = 0.289$.

A título de comparação com o procedimento bootstrap, a estatística (estimador) $\hat{\zeta}$ foi estimado em cada uma das $B = 3200$ amostras bootstrap. O desvio padrão das réplicas bootstrap

resultou 0.290 (muito próximo ao valor teórico 0.289). Histogramas para as correlações $\hat{\theta}^*$ e para os $\hat{\zeta}^*$.

! Importante

Muitas fórmulas para os erros padrões são aproximações baseadas na teoria normal e isso “explica” os resultados próximos obtidos com o uso do bootstrap paramétrico que extrai amostras a partir da distribuição normal.

Vantagens do bootstrap sobre os métodos tradicionais:

- **bootstrap não-paramétrico:** não é necessário fazer suposições de modelos paramétricos para a população;
- **bootstrap paramétrico:** possibilita estimar erros padrões em problemas para os quais não há fórmulas para os erros padrões.

5.1.4 Intervalos de Confiança

! Notação

- $\tilde{X} = (X_1, x_2, \dots, X_n)$: amostra aleatória de uma f.d.a. F
- $\tilde{x} = (x_1, x_2, \dots, x_n)$: amostra aleatória observada de F
- $\Theta = t(F)$: parâmetro (Θ uma função de F)
- $\hat{\Theta} = s(\tilde{X})$: estimador para Θ
- $\hat{\theta} = s(\tilde{x})$: estimativa para Θ

5.1.4.1 Intervalos de Confiança Normal Padrão

Suponha que $\hat{\Theta} \sim N(\Theta, ep(\hat{\Theta})^2)$. Portanto, se $ep(\hat{\Theta})$ é conhecido temos que:

$$Z = \frac{\hat{\Theta} - \Theta}{ep(\hat{\Theta})} \sim N(0, 1),$$

e

$$IC_z(100(1 - \alpha)\%, \Theta) = \hat{\Theta} \pm z_{\frac{\alpha}{2}} ep(\hat{\Theta}).$$

5.1.4.2 Intervalo de Confiança t-Student

Suponha que $\hat{\Theta} \sim N(\Theta, ep(\hat{\Theta})^2)$ e se $ep(\hat{\Theta})$ é desconhecido, portanto

$$Z = \frac{\hat{\Theta} - \Theta}{\hat{ep}(\hat{\Theta})} \sim \text{t-Student}(n-1),$$

e

$$IC_t(100(1-\alpha)\%, \Theta) = \hat{\Theta} \pm t_{(n-1, \frac{\alpha}{2})} \hat{ep}(\hat{\Theta}).$$

5.1.4.3 IC bootstrap-t

Agora vamos definir

$$Z^{*b} = \frac{\hat{\Theta}^{*b} - \hat{\Theta}}{\hat{ep}^{*b}},$$

em que $\hat{\Theta}^{*b} = s(\tilde{X}^{*b})$ e $\hat{ep}^{*b}$ são obtidos para cada amostra bootstrap $\tilde{x}^*$. O percentil $100 \cdot \alpha$ de Z^{*b} é estimado pelo valor $\hat{t}^{(\alpha)}$, tal que

$$\#\{Z^{*b} \leq \hat{t}^{(\alpha)}\}/B = \alpha.$$

Por exemplo, se $B = 100$ e a confiança para o intervalo é de 90%, a estimativa $\hat{t}^{(0.05)}$ será o quinto maior valor de Z^{*b} e a estimativa $\hat{t}^{(0.95)}$ será o nonagésimo quinto maior valor de Z^{*b} .

Este procedimento estima a distribuição de Z (quantidade pivotal) diretamente dos dados. Não é necessário a suposição teórica de normalidade e sua distribuição é a mesma para qualquer θ . O intervalo (bilateral) bootstrap-t de confiança $100 \times (1-2\alpha)\%$ para o parâmetro Θ , denotado por $IC_t^*(100(1-2\alpha)\%, \Theta)$, é definido por:

$$[\hat{\Theta} - \hat{t}^{(1-\alpha)} \hat{ep}(\hat{\Theta}), \hat{\Theta} - \hat{t}^{(\alpha)} \hat{ep}(\hat{\Theta})].$$

Emprega-se estimativas *plug-in* para $\hat{\Theta}$ e $\hat{ep}(\hat{\Theta})$; não sendo possível, o erro padrão é estimado usando bootstrap ou jackknife.

! Importante

Se $B \cdot \alpha$ não resultar um número inteiro?

1. Efron & Tibshirani (1993): supondo $\alpha \leq 0.5$, faça $k = \text{Int}[(B+1) \cdot \alpha]$ (o maior

inteiro $\leq (B+1) \cdot \alpha$) e defina os percentis empíricos de ordem $100 \cdot \alpha$ e $100 \cdot (1 - \alpha)$, respectivamente, pelo k -ésimo e $(B+1-k)$ -ésimo maiores valores de Z^{*b} .

2. Davidson & Hinkley (1997): interpolação dos percentis.

Considerações

- Este intervalo pode ser fortemente influenciado por poucos valores discrepantes.
- Em geral, é necessário um valor de B muito superior a 200.
- Para grandes amostras, o IC bootstrap- t pode estar mais próximo do nível de cobertura desejado do que os IC Normal ou t -Student.
- Abaixo, percentis da distribuição t -Student com 8 gl, normal padrão e distribuição bootstrap de Z^{*b} (para o grupo controle no experimento com os ratos; $B = 1000$).

Tabela 5.5: Comparação dos percentis de diferentes distribuições.

percentil	5	10	90	95
t_8	-1.86	-1.40	1.40	1.86
Normal	-1.65	-1.28	1.28	1.65
bootstrap- t	-4.53	-2.01	1.19	1.53

O IC bootstrap- t para Θ (média do tempo de sobrevivência de ratos não submetidos ao tratamento (grupo controle)):

$$[56.22 - 1.53 \times 13.33, 56.22 + 4.53 \times 13.33] = [35.82, 116.74],$$

usado a estimativa *plug-in* para o erro padrão da média.

Considerações Finais

- os valores dos percentis bootstrap- t podem não ser simétricos em relação ao zero.
- aplicável para estatísticas de localização (média, mediana etc).
- Cautela no uso envolvendo pequenas amostras (limites do intervalo fora do espaço paramétrico).
- o IC_t^* não é *transformation-respecting* (ver próxima seção)

5.1.4.4 Intervalos de Confiança bootstrap percentil

O intervalo bootstrap percentil de confiança $100(1-2\alpha)\%$, denotado por $IC_p^*(100(1-2\alpha)\%, \Theta)$, é definido por

$$[\hat{\Theta}^{*(\alpha)}, \hat{\Theta}^{*(1-\alpha)}],$$

em que $\hat{\Theta}^{*(\alpha)}$ é o percentil de ordem 100α da distribuição bootstrap de $\hat{\Theta}^*$. Esta definição refere-se a um número infinito de réplicas.

Em situações práticas, o intervalo é aproximado com base nos resultados de B réplicas bootstrap:

$$[\hat{\Theta}_B^{*(\alpha)}, \hat{\Theta}_B^{*(1-\alpha)}],$$

em que $\hat{\Theta}_B^{*(\alpha)}$ é dado pelo percentil de ordem 100α dos valores de $\hat{\theta}_B^{*(b)}$, isto é, o $(B \cdot \alpha)$ -ésimo valor ordenado da lista das B réplicas de $\hat{\theta}_B^{*(b)}$.

i Nota

Se a distribuição de $\hat{\Theta}^*$ é aproximadamente normal, os intervalos percentil e normal padrão são próximos.

Exemplo

Para uma amostra: $X_1, X_2, \dots, X_n, n = 10$, de uma distribuição (população) normal padrão, considere o parâmetro de interesse Θ . $\Theta = \exp(\mu)$, sendo $\mu = 0$. Sendo a estimativa para Θ : $\hat{\theta} = \exp(\bar{x})$. Além disso, use o bootstrap não paramétrico: $B = 1000$ réplicas $\hat{\theta}^*$ usando a amostra observada: (1.669, -0.411, -0.322, 0.746, -0.868, -0.874, 1.011, -0.173, 0.021, 1.482). Resultando na estimativa para Θ : $\hat{\theta} = \exp(\bar{x}) = 1.256$.

Tabela 5.6: Percentis de $\hat{\theta}^*$ com base em 1000 réplicas bootstrap.

2.5%	5%	10%	16%	50%	84%	90%	95%	97.5%
0.74	0.81	0.89	0.95	1.25	1.65	1.77	2.00	2.13

Desta maneira, temos que

$$IC_p^*(95\%, \Theta) = [0.74, 2.13],$$

e

$$IC_p^*(90\%, \Theta) = [0.81, 2.00].$$

Abaixo um código R para o exemplo.

```
set.seed(1234)
n <- 10
x <- rnorm(n) # amostra observada
B <- 1000     # no. de réplicas bootstrap
theta <- 0
alfa <- c(0.025, 0.05, 0.10, 0.16, 0.50, 0.84, 0.90, 0.95, 0.975)

for(i in 1:B){
  a <- sample(x, n, replace=TRUE) # amostra bootstrap
  theta[i] <- exp(mean(a))        # réplica bootstrap
}

exp(mean(x)) # estimativa
```

```
[1] 0.6817056
```

```
sz <- sort(theta)
rbind(100*alfa, round(sz[c(B*alfa)], 2))
```

```
      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9]
[1,]  2.50  5.00 10.00 16.00 50.00 84.00  90 95.0 97.5
[2,]  0.37  0.42  0.48  0.52  0.71  0.93   1  1.1  1.2
```

Agora iremos considerar outra abordagem. Seja $\Phi = \log(\Theta)$ e $\hat{\Phi} = \log(\hat{\Theta}) = \bar{X}$. Usando $IC_z(95\%, \Phi) = \hat{\Phi} \pm z_{\frac{\alpha}{2}} \hat{e}p(\hat{\Phi})$, resulta $[0.228 \pm 1.96 \times 0.28] = [-0.32, 0.78]$, em que $\hat{e}p(\hat{\Phi})$ é o estimador *plug-in* para o erro padrão de $\hat{\Phi}$. Fazendo a transformação inversa, o intervalo para Θ é $[0.73, 2.18]$. Note que o intervalo percentil para Θ assemelha-se ao intervalo normal padrão construído para uma transformação apropriada de Θ e transformado para a escala de Θ .

! Importante

Vantagem do método percentil: Não é necessário conhecer a transformação (normalizadora). O método percentil incorpora automaticamente a transformação adequada.

Lema

Suponha a transformação $\hat{\Phi} = m(\hat{\Theta})$ e, conseqüentemente, $\hat{\Phi} \sim N(\Phi, c^2)$, para algum erro padrão c . Então, o intervalo percentil para Θ será dado por

$$\left[m^{-1}(\hat{\Phi} - z_{\frac{\alpha}{2}} \cdot c), m^{-1}(\hat{\Phi} + z_{\frac{\alpha}{2}} \cdot c) \right].$$

Nota

- Em situações nas quais o uso de IC padrão é adequado*, o IC percentil produz resultados próximos ao do IC padrão.
- Em situações nas quais o uso do IC padrão é adequado apenas após uma transformação no parâmetro, a aplicação do método percentil automaticamente contempla essa transformação.

Considerações Finais

- Propriedade *transformation-respecting*: o intervalo percentil para qualquer transformação (monotônica) $\Phi = m(\Theta)$ do parâmetro Θ é dado por

$$\left[m(\hat{\Theta}^{*(\alpha)}), m(\hat{\Theta}^{*(1-\alpha)}) \right].$$

- A propriedade também é válida para o intervalo aproximado com base nos resultados de B réplicas bootstrap:

$$\left[m(\hat{\Theta}_B^{*(\alpha)}), m(\hat{\Theta}_B^{*(1-\alpha)}) \right].$$

- Propriedade *range-preserving*: produz intervalos com limites dentro do espaço paramétrico.

5.1.4.5 Intervalos de Confiança bootstrap - versões aprimoradas

O intervalo bootstrap- t apresenta boa probabilidade de cobertura teórica, mas tende a ser irregular na prática. Já o intervalo bootstrap percentil é menos irregular, mas apresenta probabilidade de cobertura menos satisfatória.

Proposta: intervalo bootstrap BC_a (*bias-corrected and accelerated*) e ABC (*approximate bootstrap confidence*).

substancial melhoria na prática e na teoria. correção de viés do estimador. o ABC exige menos esforço computacional do que o BC_a

5.2 O Método Jackknife

O Jackknife é um precursor histórico do Bootstrap, proposto inicialmente por Maurice Quenouille (1949) para estimativa de viés e expandido por John Tukey (1958) para estimativa de variância. Enquanto o Bootstrap utiliza sorteios aleatórios com reposição, o Jackknife adota uma abordagem sistemática, baseada na exclusão de uma observação por vez.

5.2.1 O Princípio “Leave-One-Out”

Dada uma amostra original $\mathbf{x} = (x_1, x_2, \dots, x_n)$, criamos n subamostras de tamanho $n - 1$, onde cada amostra $\mathbf{x}_{(i)}$ é obtida removendo-se a i -ésima observação. Para cada subamostra, calculamos o estimador de interesse, gerando as chamadas réplicas jackknife:

$$\hat{\theta}_{(i)} = g(\mathbf{x}_{(i)}), \quad i = 1, \dots, n.$$

5.2.2 Estimativa de Viés e Variância

Diferente do Bootstrap, que trata a distribuição empírica de forma direta, o Jackknife utiliza as réplicas para aproximar a influência de cada ponto na estatística.

1. Estimativa Jackknife do Viés: O viés do estimador original $\hat{\theta}$ é estimado por:

$$\widehat{\text{viés}}_{\text{jack}} = (n - 1)(\bar{\theta}_{\text{jack}} - \hat{\theta}),$$

em que $\bar{\theta}_{\text{jack}} = \frac{1}{n} \sum_{i=1}^n \hat{\theta}_{(i)}$.

2. Estimativa Jackknife da Variância: A variância de $\hat{\theta}$ é calculada pela dispersão das réplicas em relação à sua média:

$$\widehat{\text{var}}_{\text{jack}} = \frac{(n - 1)}{n} \sum_{i=1}^n (\hat{\theta}_{(i)} - \bar{\theta}_{\text{jack}})^2.$$

Algumas considerações importantes:

- Bootstrap: amostragem aleatória com reposição;
- Jackknife: amostras fixas;
- Jackknife requer o cálculo do estimador apenas para n amostras;

- A acurácia do estimador jackknife do erro padrão depende de quão próximo o estimador é da linearidade. Para funções fortemente não lineares, o jackknife pode ser ineficiente;
- Estimador linear: $\hat{\theta} = s(X) = \mu + \frac{1}{n} \sum_{i=1}^n \alpha(X_i)$.

Exemplo

Considere a amostra observada: $\tilde{x} = (10, 26, 30, 40, 48)$ e o estimador: $\text{mediana}(\hat{\Theta})$. As amostras e réplicas jackknife são dadas, respectivamente, por:

1. $x_{\sim(1)} = (26, 30, 40, 48)$ e $\hat{\theta}_{(1)} = 35$;
2. $x_{\sim(2)} = (10, 30, 40, 48)$ e $\hat{\theta}_{(2)} = 35$;
3. $x_{\sim(3)} = (10, 26, 40, 48)$ e $\hat{\theta}_{(3)} = 33$;
4. $x_{\sim(4)} = (10, 26, 30, 48)$ e $\hat{\theta}_{(4)} = 28$;
5. $x_{\sim(5)} = (10, 26, 30, 40)$ e $\hat{\theta}_{(5)} = 28$.

Desta forma, a estimativa jackknife do erro padrão de $\hat{\theta}$:

$$\widehat{ep}_{jack} = \left[\frac{4}{5} \sum_{i=1}^5 (\hat{\theta}_{(i)} - \bar{\theta}_{jack})^2 \right]^{\frac{1}{2}} \approx 6.38,$$

em que $\bar{\theta}_{jack} = \sum_{i=1}^5 \frac{\hat{\theta}_{(i)}}{5} = 31.8$.

5.2.3 Exemplos Computacionais em R

Diferente do Bootstrap, que exige o uso de `sample(..., replace = TRUE)`, o Jackknife é implementado através da exclusão de índices (`data[-i]`).

Exemplo 1: Estimando o Viés da Variância Amostral

Sabemos que o estimador da variância que divide pela soma por n (em vez de $n-1$) é viesado. Vamos usar o Jackknife para detectar esse viés.

```
# Dados de exemplo
set.seed(42)
dados <- rnorm(20, mean = 10, sd = 2)
n <- length(dados)
```

```
# Estimador viesado da variância (divisão por n)
var_viesada <- function(x) sum((x - mean(x))^2) / length(x)
theta_hat <- var_viesada(dados)

# Réplicas Jackknife
jack_reps <- numeric(n)
for(i in 1:n) {
  jack_reps[i] <- var_viesada(dados[-i])
}

# Cálculo do Viés Jackknife
theta_jack_bar <- mean(jack_reps)
bias_jack <- (n - 1) * (theta_jack_bar - theta_hat)

cat("Estimativa original:", theta_hat, "\n")
```

Estimativa original: 6.547375

```
cat("Viés estimado via Jackknife:", bias_jack, "\n")
```

Viés estimado via Jackknife: -0.3445987

```
cat("Estimativa corrigida (Jackknife):", theta_hat - bias_jack, "\n")
```

Estimativa corrigida (Jackknife): 6.891974

Exemplo 2: Variância da Mediana

O Jackknife é útil para estimadores onde não temos uma fórmula de variância analítica simples, como a mediana.

```
# Dados originais
x <- c(56, 101, 78, 67, 93, 87, 64, 72, 80, 69)
n <- length(x)

# Réplicas para a mediana
jack_med <- numeric(n)
for(i in 1:n) {
  jack_med[i] <- median(x[-i])
}
```

```
# Estimativa da Variância Jackknife
med_jack_bar <- mean(jack_med)
var_jack_med <- ((n - 1) / n) * sum((jack_med - med_jack_bar)^2)
se_jack_med <- sqrt(var_jack_med)

cat("Mediana Amostral:", median(x), "\n")
```

Mediana Amostral: 75

```
cat("Erro Padrão Jackknife da Mediana:", se_jack_med, "\n")
```

Erro Padrão Jackknife da Mediana: 9

5.2.4 Jackknife vs. Bootstrap

1. Sistemático vs. Aleatório: O Jackknife produz sempre o mesmo resultado para o mesmo dataset, enquanto o Bootstrap depende do número de simulações B e da semente aleatória.
2. Custo Computacional: O Jackknife exige exatamente n iterações. O Bootstrap geralmente exige $B \geq 1000$ para precisão.
3. Limitação: O Jackknife falha com estatísticas não suaves (como a mediana em amostras muito pequenas), onde o Bootstrap tende a ser mais robusto.

6 Otimização Numérica

6.1 Método de Newton

Propósito: maximizar ou minimizar a função $f(\theta)$ ou, de forma equivalente, encontrar a raiz da equação $g(\theta) = 0$, em que $g(\theta) = f'(\theta)$.

Considere um valor inicial θ_0 , próximo de $\hat{\theta}$. A expansão de Taylor de $g(\theta)$ em torno de θ_0 é:

$$g(\theta) = g(\theta_0) + g'(\theta_0)(\theta - \theta_0) + \frac{g''(\theta_0)(\theta - \theta_0)^2}{2!} + \dots$$

Para $|\theta - \theta_0|$ pequeno, desprezamos termos quadráticos ou superiores:

$$g(\theta) \approx g(\theta_0) + g'(\theta_0)(\theta - \theta_0).$$

Estamos aproximando $g(\theta)$ por uma função linear de θ que tem o mesmo valor e inclinação de $g(\theta)$ no ponto $\theta = \theta_0$.

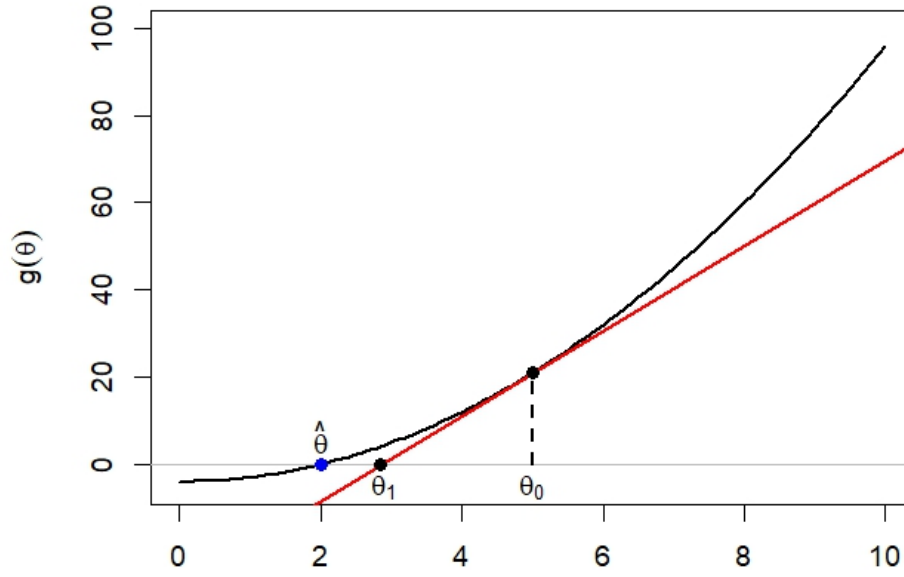


Figura 6.1: Aproximação de $g(\cdot)$ por uma função linear

O método de Newton trata com a equação da reta que passa pelo ponto $(\theta_i, g(\theta_i))$ e é tangente à curva nesse ponto:

$$g(\theta) - g(\theta_i) = g'(\theta_i)(\theta - \theta_i), \quad i = 0, 1, 2, \dots$$

em que $g'(\theta_i)$ é a inclinação da reta (derivada). Essa reta passa pelo ponto $(\theta_{i+1}, 0)$. Assim,

$$0 - g(\theta_i) = g'(\theta_i)(\theta_{i+1} - \theta_i) \Rightarrow \theta_{i+1} = \theta_i - \frac{g(\theta_i)}{g'(\theta_i)}$$

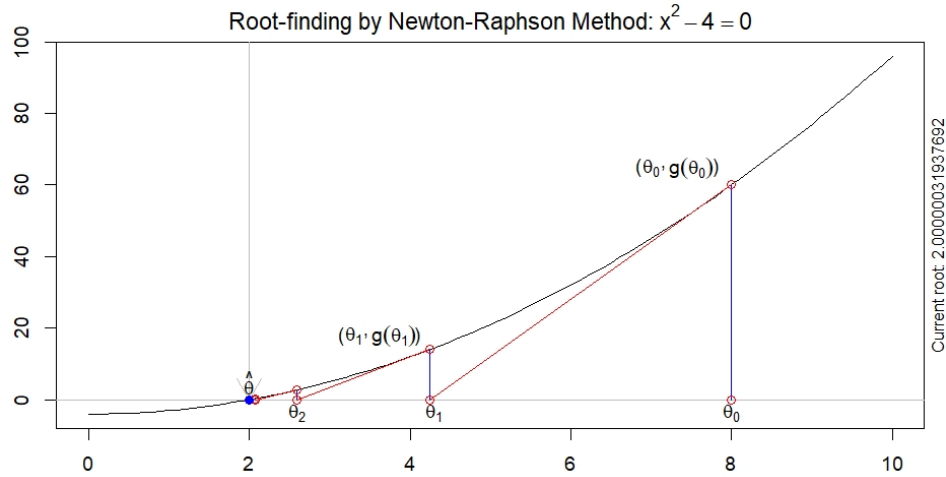


Figura 6.2: Método de Newton - Ilustração

Note que θ_0 é apenas uma “estimativa qualquer” para a raiz $\hat{\theta}$ da equação $g(\theta)$. A proposta do método é atualizar esse valor proposto da seguinte forma:

- determinar $\theta_1 = \theta_0 - \frac{g(\theta_0)}{g'(\theta_0)}$
- determinar $\theta_2 = \theta_1 - \frac{g(\theta_1)}{g'(\theta_1)}$
- repetir esse processo até que $\theta_{i+1} \approx \theta_i$
- θ_i será a raiz da equação.

O método de Newton está implementado na função `maxNR` do pacote `maxLik` do programa R.

Exemplo: Seja $f(\theta) = 100 \log(\theta) - 50\theta - 50 \log(1 - \exp(-\theta))$.

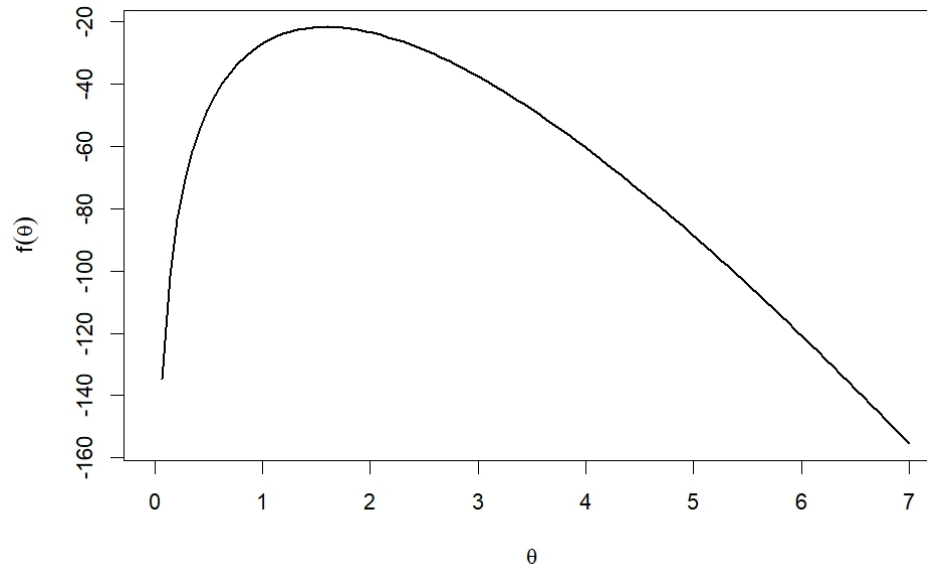


Figura 6.3: Curva da função $f(\theta)$

Determinando o ponto de máximo da função $f(\theta)$:

```
library(maxLik)

f <- function(u) (100*log(u) - 50*u-50*log(1-exp(-u)))

maxNR(f, star = 1)$estimate # 1.593624
```

```
[1] 1.593624
```

```
maxNR(f, star = 1)$maximum # -21.72331
```

```
[1] -21.72331
```

Outras funções relacionadas a otimização:

1. `optimize`: determinar o ponto de máximo (mínimo) da função $f(\theta)$:

```
f <- function(u) (100*log(u) - 50*u-50*log(1-exp(-u)))
optimize(f, lower = 0, upper = 5, maximum = TRUE)
```



```
$maximum  
[1] 1.593607
```

```
$objective  
[1] -21.72331
```

2. uniroot: determinar a raiz da equação $\frac{\partial f}{\partial \theta} = 0$.

```
f <- function(u) 100*(1/u)-50-50*(exp(-u)/(1-exp(-u)))  
uniroot(f, interval = c(0.001, 5))
```

```
$root  
[1] 1.593632
```

```
$f.root  
[1] -0.0001788499
```

```
$iter  
[1] 8
```

```
$init.it  
[1] NA
```

```
$estim.prec  
[1] 6.103516e-05
```

A função D é usada para a obtenção de *derivadas*.

```
# Exemplo 1  
f <- expression(x^4, "x")  
d1 <- D(f, "x");d1 # primeira derivada
```

```
4 * x^3
```

```
d2 <- D(d1, "x");d2 # segunda derivada
```

```
4 * (3 * x^2)
```

```
# Exemplo 2
f <- expression(100*log(u) - 50*u-50*log(1-exp(-u)))
d1 <- D(f, "u");d1 # primeira derivada
```

$$100 * (1/u) - 50 - 50 * (\exp(-u)/(1 - \exp(-u)))$$

```
d2 <- D(d1, "u");d2 # segunda derivada
```

$$-(100 * (1/u^2) - 50 * (\exp(-u)/(1 - \exp(-u)) + \exp(-u) * \exp(-u)/(1 - \exp(-u))^2))$$

```
# Exemplo 3: função para derivadas de ordem n
Dn <- function(expressao, variavel, ordem = 1) {
  if(ordem < 1) stop("ordem deve ser >= 1")
  else if(ordem == 1) D(expressao, variavel)
  else Dn(D(expressao, variavel), variavel, ordem - 1)
}
```

```
Dn(expression(x^4), "x", 1)
```

$$4 * x^3$$

```
Dn(expression(x^4), "x", 2)
```

$$4 * (3 * x^2)$$

```
Dn(expression(x^4), "x", 3)
```

$$4 * (3 * (2 * x))$$

```
# Exemplo 4: implementando o método de Newton
f <- function(u) 100*log(u)-50*u-50*log(1-exp(-u))
d1 <- function(u) (100/u)-50-(50*exp(-u))/(1-exp(-u))
d2 <- function(u) -(100/u^2)-(50*exp(u))/(exp(u)-1)^2

u <- 1 # estimativa/chute inicial
dif <- 1; i <- 2; f.max <- 0
while (dif > 0.00001) {
```

```

u[i] <- u[i-1] - d1(u[i-1])/d2(u[i-1])
dif <- abs(u[i] - u[i-1])
f.max <- f(u[i])
i <- i + 1
}

u[i-1]

```

```
[1] 1.593615
```

```
f.max
```

```
[1] -21.72331
```

Considere que $\hat{\theta}$, o estimador de máxima verossimilhança do parâmetro θ , pode ser obtido de $U(\theta) = 0$, ou seja, $\hat{\theta}$ é a raiz dessa equação. Portanto, o procedimento de Newton se estabelece fazendo $g(\theta) = U(\theta)$ e $g'(\theta) = U'(\theta) = -I(\theta)$ e é dado por

$$\theta_{i+1} = \theta_i + \frac{U(\theta_i)}{I(\theta_i)}, \quad i = 0, 1, 2, \dots$$

Quando $\theta_{i+1} \approx \theta_i$, o estimador de máxima verossimilhança será $\hat{\theta} = \theta_i$.

Observações:

1. Com bons chutes iniciais, 3 ou 4 iterações são suficientes para a convergência. Isso se deve ao fato de que para grandes amostras, $U(\theta)$ é próxima da linearidade em θ .
2. Se $U(\theta)$ é exatamente linear em θ , o método produz $\hat{\theta}$ em uma única iteração.
3. Se $U(\theta) = 0$ tem mais de uma raiz, o método não necessariamente converge para a raiz desejada. Dificuldades também surgem se o máximo ocorre no limite ou próximo a um limite do espaço paramétrico. Sugere-se examinar o gráfico de $l(\theta)$ antes de aplicar o método de Newton.

7 Algoritmo EM

8 Aplicações Modernas em Estatística Computacional

Referências

FALK, Ruma. [A Closer Look at the Notorious Birthday Coincidences](#). **Teaching Statistics**, v. 36, n. 2, p. 41–46, jun. 2014.

HODGSON, Ted; BURKE, Maurice. [On Simulation and the Teaching of Statistics](#). **Teaching Statistics**, v. 22, n. 3, p. 91–96, out. 2000.

MARTINS, Rui Manuel Da Costa. [Learning the principles of simulation using the birthday problem](#). **Teaching Statistics**, v. 40, n. 3, p. 108–111, set. 2018.

MATTHEWS, Robert; STONES, Fiona. [Coincidences: the truth is out there](#). **Teaching Statistics**, v. 20, n. 1, p. 17–19, 1998.

THOMAS, F. H.; MOORE, J. L. [CUSUM: Computer Simulation for Statistics Teaching](#). **Teaching Statistics**, v. 2, n. 1, p. 23–28, jan. 1980.

TINTLE, Nathan *et al.* [Combating Anti-Statistical Thinking Using Simulation-Based Methods Throughout the Undergraduate Curriculum](#). **The American Statistician**, v. 69, n. 4, p. 362–370, 2 out. 2015.

ZIEFFLER, Andrew; GARFIELD, Joan B. Studying the Role of Simulation in Developing Students' Statistical Reasoning. *In*: International Statistical Institute, 2007.